

NOVA Context Master Reference

Combined Technical Documentation

32 source documents

Table of Contents

1. 00_NOVA_CONTEXT_INDEX.md
2. 01_NOVA_ARCHITECTURE.md
3. 02_NOVA_CAPABILITIES.md
4. 03_NOVA_API_INTEGRATIONS.md
5. 04_NOVA_ENVIRONMENT_CONFIG.md
6. 05_NOVA_AI_PROVIDERS.md
7. 06_NOVA_MEMORY.md
8. 07_NOVA_VOICE.md
9. 08_NOVA_BROWSER.md
10. 09_NOVA_COMPUTER_CONTROL.md
11. 10_NOVA_ENVIRONMENT_AWARENESS.md
12. 11_NOVA_TASK_EXECUTION.md
13. 12_NOVA_AUTOMATION.md
14. 13_NOVA_SAFETY.md
15. 14_NOVA_PERMISSIONS.md
16. 15_NOVA_EVENTBUS.md
17. 16_NOVA_UI.md

18. 17_NOVA_APPLICATIONS.md
19. 18_NOVA_FILESYSTEM.md
20. 19_NOVA_DEPENDENCIES.md
21. 20_NOVA_TESTING.md
22. 21_NOVA_STARTUP.md
23. 22_NOVA_SECURITY.md
24. 23_NOVA_DATA_FLOW.md
25. 24_NOVA_REUSE_GUIDE.md
26. 25_NOVA_LIMITATIONS.md
27. 26_NOVA_EXTENSION_POINTS.md
28. NOVA_API_CONTRACTS.md
29. NOVA_CAPABILITY_REGISTRY.md
30. NOVA_COMPLETE_CONTEXT.md
31. NOVA_NON_BREAKING_CONTRACT.md
32. NOVA_REUSE_MATRIX.md

1. 00_NOVA_CONTEXT_INDEX.md

Source file: 00_NOVA_CONTEXT_INDEX.md

NOVA Context Index & Navigation Guide

This directory (`docs/nova-context/`) forms the complete technical master reference for the NOVA operating system, designed to provide exhaustive, verified architectural knowledge for future autonomous AI agent integration without breaking existing working systems or duplicating capabilities.



Core Documentation Index

File	Subsystem / Topic	Description
NOVA_COMPLETE_CONTEXT.md	Master Reference	Comprehensive single-source technical reference of the entire NOVA codebase.
NOVA_REUSE_MATRIX.md	Reuse Matrix	Exhaustive classification of what to reuse directly, wrap/adapt, extend, or avoid duplicating.
NOVA_API_CONTRACTS.md	API Contracts	Concrete method signatures, parameters, return types, exceptions, and side effects.
NOVA_CAPABILITY_REGISTRY.md	Capability Catalog	End-to-end trace from Intent $\rightarrow$ Router $\rightarrow$ Handler $\rightarrow$ Executor $\rightarrow$ OS API $\rightarrow$ Verification.
NOVA_NON_BREAKING_CONTRACT.md	Invariants & Safety	Critical stability invariants, configuration schema, and compatibility rules.



Topical Deep-Dives

1. [01_NOVA_ARCHITECTURE.md](#) — Macro architecture, layers, core subsystems, lifecycle management.
2. [02_NOVA_CAPABILITIES.md](#) — Comprehensive inventory of all 35+ system, browser, desktop, visual, and task capabilities.
3. [03_NOVA_API_INTEGRATIONS.md](#) — External cloud APIs, SDK clients, auth methods, rate limiting, and fallbacks.
4. [04_NOVA_ENVIRONMENT_CONFIG.md](#) — Central settings schema, environment variables, validation rules, secrets safety.
5. [05_NOVA_AI_PROVIDERS.md](#) — AI routing engine, Gemini/Groq/OpenRouter/Cerebras clients, failover strategy.

6. [06_NOVA_MEMORY.md](#) — Atomic JSON persistence, VectorStore semantic embeddings, breadcrumbs, search.
7. [07_NOVA_VOICE.md](#) — Voice V2 pipeline: PyAudio $\rightarrow$ Silero VAD $\rightarrow$ Faster-Whisper $\rightarrow$ Edge-TTS/Kokoro.
8. [08_NOVA_BROWSER.md](#) — Native AppleScript & URL browser automation, site-specific skills, auto-scrolling.
9. [09_NOVA_COMPUTER_CONTROL.md](#) — Mouse, keyboard, Retina coordinates, accessibility hierarchy, popup closing.
10. [10_NOVA_ENVIRONMENT_AWARENESS.md](#) — EYES perception, deictic pronoun resolution ("this", "that"), real-time state.
11. [11_NOVA_TASK_EXECUTION.md](#) — Autonomous Task Agent V3: Goal planning, step execution, observation, replanning.
12. [12_NOVA_AUTOMATION.md](#) — Proactive perception triggers, background workers, multi-step routines.
13. [13_NOVA_SAFETY.md](#) — Confirmation gates, destructive action guards, safe workspace sandboxes.
14. [14_NOVA_PERMISSIONS.md](#) — macOS Accessibility, Screen Recording, Microphone, Automation/AppleEvents.
15. [15_NOVA_EVENTBUS.md](#) — Thread-safe publish-subscribe EventBus, event catalog, payloads, lifecycles.
16. [16_NOVA_UI.md](#) — React 18, TypeScript, Tailwind, Electron desktop wrapper, WebSocket gateway.
17. [17_NOVA_APPLICATIONS.md](#) — Finder, Terminal, VS Code, Chrome, Safari, TextEdit, Photo Booth, System Settings.
18. [18_NOVA_FILESYSTEM.md](#) — Multi-root search, safe trash integration, overwrite protection, AI document generation.
19. [19_NOVA_DEPENDENCIES.md](#) — Python packages, Node dependencies, system binaries (FFmpeg, osascript).
20. [20_NOVA_TESTING.md](#) — 881-test test suite catalog, test categories, mocks, running instructions.
21. [21_NOVA_STARTUP.md](#) — Multithreaded bootstrap flow, signal handling, graceful shutdown, state serialization.
22. [22_NOVA_SECURITY.md](#) — Secret masking, `.gitignore` defense, path containment, IPC authorization.
23. [23_NOVA_DATA_FLOW.md](#) — End-to-end data flow diagrams (Voice, Text, Vision, Browser, Computer, Tasks).
24. [24_NOVA_REUSE_GUIDE.md](#) — How the future autonomous agent must interface with NOVA's capability layer.
25. [25_NOVA_LIMITATIONS.md](#) — Known hardware limitations, OS restrictions, external CLI dependencies.
26. [26_NOVA_EXTENSION_POINTS.md](#) — Preferred architecture-approved extension points for adding future features.

2. 01_NOVA_ARCHITECTURE.md

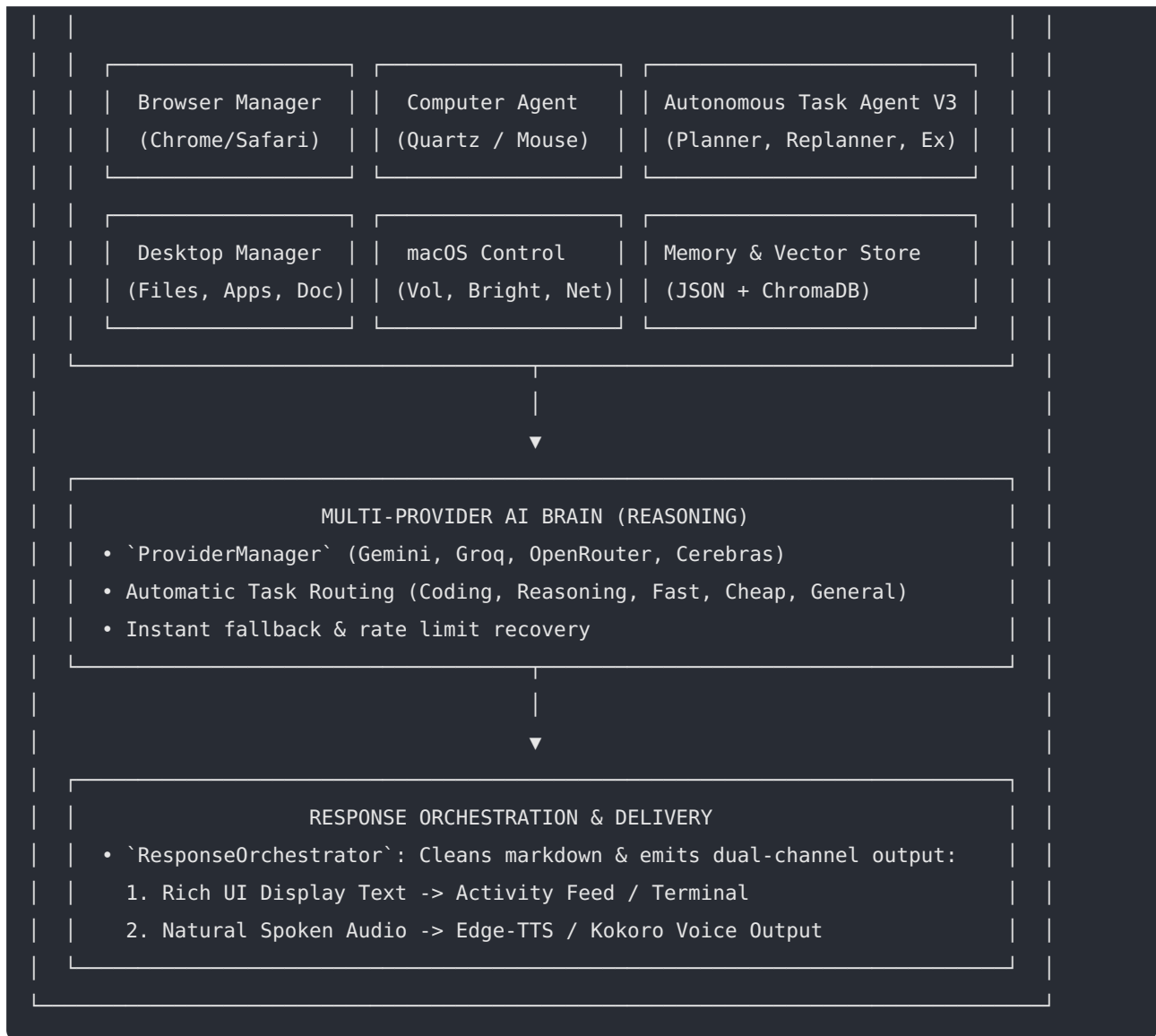
Source file: 01_NOVA_ARCHITECTURE.md

01 — NOVA Macro Architecture & Lifecycle

Architecture Overview

NOVA follows a layered, modular, event-driven architecture designed for local-first execution on macOS.





Subsystem Threading Model

- **Main Thread:** Executes the `_run_event_loop()` draining `_turn_queue`.
- **Background Worker Threads:**
 - `nova-console-input`: Listens for terminal typed commands and slash-commands.
 - `voice-listener`: Continuous PyAudio streaming and Silero VAD analysis.
 - `nova-ui-server`: Asyncio event loop running HTTP REST + WebSocket server on port 8765.
 - `nova-eyes-polling`: Passive proactive perception monitor for screen errors.
- **Locking:** All shared resources utilize `threading.RLock` or `threading.Lock` to prevent race conditions.

3. 02_NOVA_CAPABILITIES.md

Source file: 02_NOVA_CAPABILITIES.md

02 — NOVA Capabilities & Reachability Map

Overview

NOVA contains 35+ first-class capabilities accessible via text, voice, and autonomous task agent plans.

1. System Hardware Controls (`mac_control/actions/`)

- **Volume:** `increase volume`, `decrease volume`, `set volume to X%`, `mute`, `unmute`, `get volume`.
- Executed via AppleScript `set volume output volume X`.
- **Brightness:** `increase brightness`, `decrease brightness`, `set brightness to X%`, `check brightness`.
- Executed via native `brightness` CLI or AppleScript display service.
- **Wi-Fi:** `is wi-fi on`, `turn wi-fi on`, `turn wi-fi off`, `what wi-fi am i connected to`.
- Executed via macOS `networksetup -setairportpower`.
- **Bluetooth:** `check bluetooth status`, `turn bluetooth on/off`.
- Executed via `blueutil` CLI with native fallback.
- **Clipboard:** `what is in my clipboard`, `clear clipboard`, `copy this to clipboard`.
- Executed via macOS `pbcopy` and `pbpaste`.
- **Lock Screen & Shutdown:** `lock my mac`, `shut down my mac`.
- System shutdown requires explicit spoken/typed confirmation.

2. Application & Desktop Controls (`desktop/`)

- **App Launcher (`AppLauncher`):** Launches and closes applications (e.g. VS Code, Chrome, Safari, Terminal, Telegram, Spotify, TextEdit).
- **Camera (`CameraManager`):** Opens Photo Booth viewfinder and takes snapshot photos saved to `~/Pictures/NOVA`.
- **Filesystem (`FileManager`):** Safe multi-root search across Desktop, Documents, Downloads, Projects, and Workspace. Creates folders, creates files, and safely moves items to macOS Trash.
- **AI Document Editor (`DocumentEditor`):** Uses AI Provider to generate structured notes/ essays and automatically creates and opens them in TextEdit.

3. Native Browser Automation (`browser/`)

- **Browser Engines:** Chrome and Safari via `MacOSNativeBrowserEngine` (AppleScript).

- **Navigation & Search:** open <url>, search Google for <query>, search <site> for <query>.
- **Tab Management:** open a new tab, search inside this tab, close tab, switch to tab N.
- **Media Skills:** YouTube playback, search artist/song, auto-play next, YouTube Shorts auto-scrolling (YouTubeShortsScroller).
- **Web Extraction:** Reads and extracts article text, summaries, and structured content from generic webpages (WebpageExtractor).

4. NOVA Eyes & Computer Control (core/eyes/ , core/computer_agent.py)

- **Vision & OCR:** In-memory Retina screen grabs (CGWindowListCreateImage) with Apple Vision framework OCR (VNRecognizeTextRequest).
- **Accessibility:** Queries active windows, UI buttons, and input fields via AXUIElement .
- **UI Interaction:** Clicks UI buttons by label, types text into active focus, closes popups, and scrolls viewports until target text is found.
- **Screen Understanding:** what am I looking at , explain this error , summarize this page , what is at my cursor .

5. Multi-Step Task Planning (core/task_agent/)

- **Task Planner:** Decomposes complex natural language instructions into ordered steps.
- **Task Executor:** Executes steps sequentially with verification and cancellation support.
- **Dynamic Replanner:** Automatically analyzes errors, inspects screen state, and generates alternative plans when steps fail.

4. 03_NOVA_API_INTEGRATIONS.md

Source file: 03_NOVA_API_INTEGRATIONS.md

03 — NOVA API Integrations & External Clients

Overview

NOVA integrates with multiple external cloud AI services, web data providers, and native OS SDKs. All keys are loaded securely via `config.settings.settings` and validated at startup.

Cloud AI & Service Providers

1. Google Gemini

- **SDK:** `google-genai>=1.2.0` (`google.genai.Client`)
- **Config Key:** `GEMINI_API_KEY`
- **Default Model:** `gemini-2.5-flash`
- **Endpoints:** `generativelanguage.googleapis.com:443`
- **Capabilities:** High-speed streaming, structured JSON output, vision/multimodal reasoning, tool calling.
- **Failover Role:** Primary reasoning & coding provider in NOVA.

2. Groq

- **SDK:** `groq>=0.11.0` (`groq.Groq`)
- **Config Key:** `GROQ_API_KEY`
- **Default Model:** `groq/compound-mini` (or Llama 3 models)
- **Endpoints:** `api.groq.com:443`
- **Capabilities:** Ultra-low latency inference for quick conversational turns and fast mode tasks.
- **Failover Role:** Secondary fast provider.

3. OpenRouter

- **SDK:** `openai>=1.50.0` (`openai.OpenAI(base_url="https://openrouter.ai/api/v1")`)
- **Config Key:** `OPENROUTER_API_KEY`
- **Default Model:** `openai/gpt-4o-mini`
- **Endpoints:** `openrouter.ai:443`
- **Capabilities:** Broad multi-model access and backup reasoning.
- **Failover Role:** Secondary reasoning provider.

4. Cerebras Cloud

- **SDK:** `cerebras-cloud-sdk>=1.0.0` (`cerebras.cloud.sdk.Cerebras`)
- **Config Key:** `CEREBRAS_API_KEY`
- **Default Model:** `qwen-3.8-27b`
- **Endpoints:** `api.cerebras.ai:443`
- **Capabilities:** High-throughput low-cost inference.
- **Failover Role:** Primary cheap mode provider.

5. Web Search & Data Providers

- **Tavily:** `TAVILY_API_KEY` — Deep web search API for web research planner.
- **ElevenLabs:** `ELEVENLABS_API_KEY` — Optional high-fidelity neural voice synthesis.
- **OpenWeatherMap:** `OPENWEATHER_API_KEY` — Real-time weather diagnostics.
- **NewsAPI:** `NEWS_API_KEY` — Current news and media retrieval.
- **Google Maps:** `GOOGLE_MAPS_API_KEY` (Optional) — Location and navigation services.



Native macOS System Bridges

- **AppleScript (`osascript`):** Used for Chrome/Safari browser tab manipulation, Finder queries, and system automation.
- **Quartz CoreGraphics (`pyobjc`):** In-memory Retina display capture (`CGWindowListCreateImage`) and hardware mouse/keyboard event injection (`CGEvent`).
- **Apple Vision Framework:** Native macOS OCR (`VNRecognizeTextRequest`) running on Apple Silicon Neural Engine.
- **PortAudio (`PyAudio`):** Low-latency 16kHz microphone stream capture.

5. 04_NOVA_ENVIRONMENT_CONFIG.md

Source file: 04_NOVA_ENVIRONMENT_CONFIG.md

04 — NOVA Environment Configuration & Settings

Central Configuration System

NOVA manages all configuration through a centralized, validated Pydantic Settings v2 model defined in `config/settings.py`.

Environment Variable Catalog

Variable Name	Type	Required	Default Value	Description
GEMINI_API_KEY	SecretStr	Yes	—	Google Gemini API key for primary reasoning
GROQ_API_KEY	SecretStr	Yes	—	Groq API key for fast inference
OPENROUTER_API_KEY	SecretStr	Yes	—	OpenRouter API key for backup model access
CEREBRAS_API_KEY	SecretStr	Yes	—	Cerebras Cloud API key for cheap/fast inference
TAVILY_API_KEY	SecretStr	Yes	—	Tavily Search API key for deep web research
ELEVENLABS_API_KEY	SecretStr	Yes	—	ElevenLabs API key for optional neural speech
OPENWEATHER_API_KEY	SecretStr	Yes	—	OpenWeatherMap API key
NEWS_API_KEY	SecretStr	Yes	—	NewsAPI key for news digests
GOOGLE_MAPS_API_KEY	SecretStr	No	None	Optional Google Maps API key
APP_ENV	str	No	"development"	Environment (development ,

Variable Name	Type	Required	Default Value	Description
				production , test)
DEBUG	bool	No	False	Debug mode toggle
LOG_LEVEL	str	No	"INFO"	Logging level (DEBUG , INFO , WARNING , ERROR)
STT_ENGINE	str	No	"whisper"	Speech-to-text engine (whisper , faster_whisper)
TTS_ENGINE	str	No	"edge_tts"	Text-to-speech engine (edge_tts , pyttsx3 , kokoro)
TTS_VOICE	str	No	"hi-IN-SwaraNeural"	Voice model identifier for TTS
WHISPER_MODEL_NAME	str	No	"small"	Whisper model size (tiny , base , small , medium)
STT_LANGUAGE	str	No	"auto"	ASR language code (auto , en , hi)
DEFAULT_BROWSER	str	No	"chrome"	Default web browser (chrome , safari)
DESKTOP_WORKSPACE_DIR	str	No	"~/Desktop/NOVA_WORKSPACE"	Default sandboxed file workspace
DESKTOP_CAMERA_OUTPUT_DIR	str	No	"~/Pictures/NOVA"	Destination folder for camera snapshots
NOVA_BROWSER_DEBUG	bool	No	False	Enable browser action debug logging
NOVA_INTENT_DEBUG	bool	No	False	Enable intent parsing telemetry
NOVA_MEDIA_DEBUG	bool	No	False	Enable media matching debug output
NOVA_ENVIRONMENT_DEBUG	bool	No	False	Enable environment observer debug output
NOVA_COMPUTER_AGENT_DEBUG	bool	No	False	

Variable Name	Type	Required	Default Value	Description
				Enable computer agent perception logs
NOVA_TASK_AGENT_DEBUG	bool	No	False	Enable task agent step execution logs

Security Best Practices

- **No Plaintext Secrets in Logs:** All keys use Pydantic's `SecretStr` to prevent accidental serialization.
- **Git Ignore Protection:** `.env` and `credentials.json` are excluded in `.gitignore`.
- **Doctor Verification:** `ui/doctor.py` checks for the presence and non-emptiness of required keys without ever printing the key strings.

6. 05_NOVA_AI_PROVIDERS.md

Source file: 05_NOVA_AI_PROVIDERS.md

05 — NOVA Multi-Provider AI Architecture

Overview

NOVA's AI brain is managed by `providers.provider_manager.ProviderManager`, a resilient multi-model orchestrator that isolates provider mechanics behind `providers.base_provider.BaseProvider`.

AI Request Flow

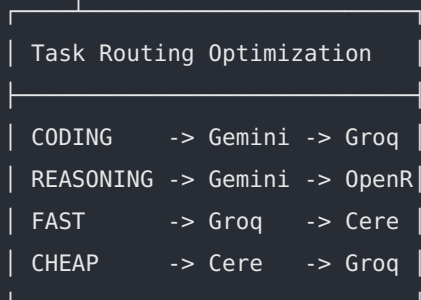
User Text / Turn Request



System Prompt & Personality Construction (``SystemPromptManager`` + ``EmotionEngine``)



``ProviderManager.generate_response(prompt, system_prompt, task_type)``



Execute Request on Candidate Provider

└─> Success: Return response, update latency & token metrics

└─> AIProviderError: Record error, switch instantly to next provider in priority list



Sanitization via ``core.response_cleaner.clean_model_response``



Response Orchestration (``ResponseOrchestrator`` -> TTS + UI)

Provider Resiliency Features

1. **Zero-Crash Failover:** If an API endpoint experiences HTTP 429 (Rate Limit), 503 (Service Unavailable), or timeout, `ProviderManager` catches `AIProviderError` and invokes the fallback candidate automatically.
2. **Provider Health Monitoring:** Tracks per-provider consecutive failure counts, average latency, and last success timestamps.
3. **Manual Override Support:** Supports interactive debugging overrides (e.g. `/provider groq` or `/provider cerebras`).
4. **Token Budgeting:** Dynamically tunes `max_tokens` for voice turns (e.g. 120 tokens for greetings, 250 for quick answers, 500 for coding/reasoning) to minimize audio synthesis delay.

7. 06_NOVA_MEMORY.md

Source file: 06_NOVA_MEMORY.md

06 — NOVA Memory Architecture & Persistence

Overview

NOVA features a dual-layer memory system: 1. **Durable Structured Memory (MemoryManager)**: Atomic JSON-persisted facts, user preferences, project tracking, goals, and session breadcrumbs stored at `data/memory/memory_store.json`. 2. **Semantic Vector Store (VectorStore)**: Embedding-based document storage and semantic similarity search using ChromaDB (with pure in-memory fallback).



Memory Categories (MemoryCategory)

- **PROFILE** : User identity, background, and personal facts.
 - **PREFERENCES** : User settings, favorite tools, coding conventions, tone preferences.
 - **PROJECTS** : Active codebase names, repository locations, project summaries.
 - **CODING** : Coding patterns, architectural decisions, language preferences.
 - **EDUCATION** : Learning goals, study topics, problem-solving history.
 - **TASKS** : Actionable to-do items and multi-step task goals.
 - **REMINDERS** : Time-based or event-based reminders.
 - **CONVERSATIONS** : Important conversational summaries and facts.
 - **NOTES** : Freeform notes.
 - **GOALS** : Long-term objectives.
 - **AI_SETTINGS** : Custom NOVA instructions and behavior overrides.
-



Atomic Persistence & Quarantine Guard

- **Atomic Writes**: Saves are executed by writing to a temporary file in the same filesystem directory followed by an atomic replace (`os.replace`), eliminating the risk of partial corruption during crashes.
- **Corrupted Store Quarantine**: If `memory_store.json` contains invalid JSON on startup, NOVA automatically renames it to `memory_store.json.corrupt.<timestamp>` and initializes a clean store rather than crashing.
- **Cross-Session Breadcrumbs**: On shutdown, `NovaApplication` writes a session breadcrumb summary (`_BREADCRUMB_KEY = "last_session_summary"`), allowing NOVA to greet the user with full context on subsequent startups.

8. 07_NOVA_VOICE.md

Source file: 07_NOVA_VOICE.md

07 — NOVA Voice V2 Subsystem Architecture

Overview

NOVA Voice V2 provides a voice-first interaction pipeline with continuous background listening, energy-adaptive VAD, local speech recognition, acoustic non-speech event detection, quality filtering, and speech synthesis.



Audio Processing Pipeline

```
[Microphone (PyAudio)] -> 16kHz Mono Stream
|
▼
[Silero VAD / Energy Detector (`VoiceActivityDetector`)]
├─ Silence / Noise Calibration
├─ Min Speech Threshold (0.35s)
└─ Finalized Speech Segment Buffer
|
▼
[Acoustic Audio Event Detector (`AudioEventDetector`)]
├─ Intercepts non-speech events (cough, laugh, sneeze, sigh)
└─ Strips acoustic markers from transcript
|
▼
[Faster-Whisper Transcriber (`TranscriptionManager`)]
├─ Localized inference (model: small.en / base)
├─ Generates candidate hypotheses & confidence
└─ Normalizes ASR artifacts & numbers
|
▼
[Transcript Quality Gate (`TranscriptQualityGate`)]
├─ Discards phantom hallucinations (e.g. repeated "Thank you" in silence)
└─ Evaluates confidence score threshold (>0.45)
|
▼
[Local Command Recognizer (`CommandRecognizer` / `VoiceCommandRouter`)]
├─ Instant route for "shutdown", "stop", "cancel"
```

└─ Passes natural language commands to `NovaApplication.\_turn\_queue`

|

▼

[Speech Synthesizer (`SpeakerManager`)]

└─ Microsoft Edge-TTS (`hi-IN-SwaraNeural` or customizable)

└─ Pygame Audio Mixer / SoundDevice playback

└─ Interruption / barge-in support

Key Voice Invariants

1. **TTS Cleanliness:** TTS input is stripped of emojis, markdown bullets, URLs, and code blocks before speaking via `ResponseOrchestrator.clean_for_speech()`.
2. **Audio Event Empathy:** Acoustic non-speech cues (coughs, sneezes) produce instant empathetic responses ("Bless you! You okay?") without LLM latency.
3. **Graceful Degradation:** If audio hardware or PortAudio is unavailable, NOVA automatically continues in typed console / UI mode with `voice_available = False`.

9. 08_NOVA_BROWSER.md

Source file: 08_NOVA_BROWSER.md

08 — NOVA Browser Automation & Web Skills

Overview

NOVA provides native browser automation on macOS without requiring third-party browser extensions or intrusive debugging proxies. Automation is executed primarily through `MacOSNativeBrowserEngine` via AppleScript, interacting with Google Chrome and Safari.



Supported Web Skills & Integrations

1. YouTube & Media Playback (`browser/sites/youtube.py`)

- **Direct Search & Play:** `play blinding lights on youtube`, `listen to lofi beats`.
- **Auto-Play & Next:** Automatically selects the best search result and starts video playback.
- **YouTube Shorts Auto-Scroller (`browser/auto_scroll.py`):** Starts a background thread (`YouTubeShortsScroller`) that automatically scrolls through YouTube Shorts at configurable intervals (`browser_auto_shorts_interval_seconds = 15.0`).
- **Media Controls:** Supports pause, resume, scroll next, and volume adjustments.

2. Google Search (`browser/sites/google.py`)

- **Search Queries:** `search Google for quantum computing tutorials`.
- **Search Inside Current Tab:** `search within this tab for pricing`.
- **Snippet & Result Extraction:** Extracts top search titles, URLs, and snippets directly into context.

3. E-Commerce & Product Search (`browser/sites/amazon.py`)

- **Product Search:** `search Amazon for mechanical keyboards under $100`.
- **Filtering:** Filters by category, price, and customer rating.

4. Developer Search (`browser/sites/github.py`)

- **Repository Search:** `search GitHub for fast whisper macos`.
- **Direct Navigation:** Navigates directly to user/repository code trees.

5. Multi-Step Deep Web Research (`browser/planner.py`)

- **Autonomous Research Planner:** Decomposes a research question into multiple search queries, visits candidate pages, extracts text with `browser.extractor.WebpageExtractor`, and compiles a structured markdown summary report.
-



Browser Safety Policy (`browser/safety.py`)

- **Dangerous Schemes Blocked:** `javascript:`, `data:`, `file:`, `vbscript:` URLs are automatically blocked.
- **Sensitive Operations Gate:** Financial transactions, password inputs, or account deletions trigger mandatory user confirmation prompts.

10. 09_NOVA_COMPUTER_CONTROL.md

Source file: 09_NOVA_COMPUTER_CONTROL.md

09 — NOVA Computer Control & UI Interaction

Overview

NOVA's computer control subsystem (`core.computer_agent.ComputerAgent` and `core.eyes`) gives NOVA the ability to perceive and manipulate any macOS application using hardware-level events and native accessibility APIs.

Interaction Capabilities

1. Quartz CGEvent Hardware Simulation

- **Retina-Aware Coordinates:** Maps logical window coordinates to physical display pixels taking Retina scale factors (`DisplayMetrics`) into account.
- **Mouse Clicks:** Injects single click, double click, and right click via `CGEventCreateMouseEvent(None, kCGEventLeftMouseDown, (x, y), kCGMouseButtonLeft)`.
- **Keyboard Typing:** Injects keystrokes and modifier combinations (Command, Option, Shift, Control) via `CGEventCreateKeyboardEvent`.
- **Scrolling:** Smooth pixel and line scrolling using `CGEventCreateScrollWheelEvent2`.

2. High-Level Visual Goals

- `click_element(label)` : Searches current screen state via OCR and Accessibility tree, identifies target bounding box, and clicks center.
 - `type_into_focused_or_target(text, label)` : Focuses targeted input field or types directly into active cursor focus.
 - `close_popup()` : Detects dialog close buttons ("X", "Close", "Dismiss", "Cancel", "Not now") and dismisses popups.
 - `scroll_until_visible(text)` : Repeatedly scrolls active window until target text appears in OCR bounding boxes.
-

The Computer Agent Loop

1. OBSERVE -> Capture in-memory Quartz frame (``MemoryFrame``) + AXUIElement tree
2. UNDERSTAND-> Apple Vision OCR extracts text boxes; Accessibility parses UI buttons

- 3. PLAN -> Spatial & semantic resolver identifies exact target coordinates
- 4. ACT -> Injects Quartz CGEvent mouse click or keystroke sequence
- 5. VERIFY -> Re-observes screen state to confirm UI delta or dismiss confirmation

11. 10_NOVA_ENVIRONMENT_AWARENESS.md

Source file: 10_NOVA_ENVIRONMENT_AWARENESS.md

10 — NOVA Environment Awareness & Pronoun Resolution

Overview

NOVA maintains real-time perception of the user's computing context through `EnvironmentObserver` (EYES) and deterministically resolves ambiguous natural language references ("this", "that", "it", "here") through `ContextResolver`.

Observed Environment State (`EnvironmentContext`)

`EnvironmentObserver.refresh()` queries macOS via lightweight AppleScript calls (cached with 0.5s TTL) to capture: - **Active Frontmost Application:** e.g. "Google Chrome", "Visual Studio Code", "Finder", "Terminal". - **Active Window Title:** Exact title string of the front window. - **Active Browser State:** Active tab index, tab count, current URL, domain, and page title. - **Finder & Filesystem State:** Selected file/folder paths, open directory in active Finder window. - **System Media & Recording:** Active screen recording status and output file path.

Deterministic Reference Resolution (`ContextResolver`)

When the user says: - "Delete this" $\$to\$$ Resolves to currently selected file/folder in Finder, or `last_created_path`. - "Search inside this tab for pricing" $\$to\$$ Resolves to active browser URL / tab. - "Open that in VS Code" $\$to\$$ Resolves to `last_created_path` or selected Finder file. - "What is here?" $\$to\$$ Resolves to current directory or visual cursor position.

```
# Context resolution signature
resolved = ContextResolver.resolve_target(
    reference_term="this",
    context=environment_context,
    target_type="any" # "file", "url", "app", "any"
)
# Returns: {"resolved": True, "target": Path("/Users/.../file.py"), "target_type": "file"}
```

12. 11_NOVA_TASK_EXECUTION.md

Source file: 11_NOVA_TASK_EXECUTION.md

11 — Autonomous Task Agent V3 Subsystem

Overview

NOVA includes an Autonomous Task Agent (`core.task_agent`) that plans, executes, verifies, and dynamically recovers multi-step operational tasks on macOS.



Core Architecture Components

User Goal ("Create a project folder, scaffold a Python script, and search Google for docs")

|



[TaskPlanner (``core.task_agent.planner.TaskPlanner``)]

- └─ Decomposes goal into sequential ``TaskStep`` definitions
- └─ Assigns step IDs, capability names, parameter dicts, and dependencies
- └─ Returns a validated ``TaskPlan``

|



[TaskExecutor (``core.task_agent.executor.TaskExecutor``)]

- └─ Iterates through ``plan.steps`` sequentially
- └─ Step Pre-check: Validates step dependencies and cancellation flags
- └─ Capability Dispatch: Looks up capability handler in ``CapabilityRegistry``
- └─ Step Execution: Runs handler with ``TaskContext``
- └─ Step Verification: Executes capability-specific verifier function
 - └─> Step Succeeded: Updates ``TaskContext`` and advances to next step
 - └─> Step Failed: Invokes ``DynamicReplanner``

|



[DynamicReplanner (``core.task_agent.replanner.DynamicReplanner``)]

- └─ Analyzes step failure, error message, and real-time screen state
- └─ Generates alternative steps (e.g. alternate path, browser fallback)
- └─ Injects recovery steps into active plan
- └─ Allows up to ``max_replans=3`` before declaring task failure

Data Models (`core.task_agent.models`)

- **TaskGoal** : High-level description, priority, and risk rating.
- **TaskStep** : `step_id`, `name`, `capability`, `parameters`, `status` (`PENDING`, `RUNNING`, `SUCCESS`, `FAILED`, `CANCELLED`), `dependencies`, `result`, `error`.
- **TaskPlan** : `plan_id`, `goal`, `steps`, `status`, `max_steps=10`, `timeout_seconds=120.0`.
- **TaskContext** : Shared state across steps tracking created folders, files, opened apps, active URLs, and artifacts.
- **TaskResult** : Final outcome with execution time, steps completed, and spoken response.

13. 12_NOVA_AUTOMATION.md

Source file: 12_NOVA_AUTOMATION.md

12 — NOVA Automation Systems & Triggers

Overview

Automation in NOVA extends beyond manual commands to encompass proactive triggers, event-driven perception, background auto-scrolling, and multi-step background workflows.

Active Automation Subsystems

1. Proactive Perception Engine

(`core.eyes.proactive.ProactiveAssistanceEngine`)

- **Background Screen Monitor:** Periodically inspects active windows for high-urgency errors, compiler exceptions, crashed processes, or unhandled dialogs.
- **Delivery:** Dispatches non-intrusive proactive assistance cards (`ProactiveOpportunity`) to the UI activity feed without interrupting active voice speech.

2. Auto-Scroll Automation (`browser.auto_scroll.YouTubeShortsScroller`)

- **Threaded Auto-Scroller:** Executes in a dedicated background daemon thread.
- **Dynamic Controls:** Supports `start` , `pause` , `resume` , and `stop` commands via text or voice.
- **Interval Control:** Configurable via `BROWSER_AUTO_SHORTS_INTERVAL_SECONDS` .

3. Screen Recording Subsystem

(`core.screen_recording.ScreenRecordingManager`)

- **Background Capture:** Starts and stops macOS screen video recording saved to `~/Desktop/NOVA Recordings` .
- **Telemetry:** Automatically reports start/stop status to `DashboardStatsManager` and `EventBus`.

4. Background Server Daemons

- **UI Async Server:** Python `asyncio` server running on port 8765.
- **Voice Listener Worker:** Continuous audio streaming thread with VAD energy detection.

- **Console Input Worker:** Interactive terminal listener with slash-command capabilities (`/filter` , `/mode` , `/search` , `/scroll` , `/copy` , `/clear`).

14. 13_NOVA_SAFETY.md

Source file: 13_NOVA_SAFETY.md

13 — NOVA Safety Architecture & Operation Guards

Overview

NOVA enforces multi-tier safety checks across all filesystem, desktop, browser, and hardware interactions to prevent unintended damage, data loss, or unsafe execution.

Safety Mechanisms

1. Mandatory Confirmation Gates

- **Destructive File Deletion:** Any request to delete or trash a file/folder (`DELETE_ITEM`) sets `pending_confirmation = {"action": "DELETE_ITEM", "target_path": ...}` and asks the user: *"Do you want me to delete [filename]?"*
- **System Shutdown:** Any shutdown request (`SYSTEM_SHUTDOWN`) triggers a confirmation gate when voice confidence is uncertain or command is received.
- **Confirmation State Resolver:** When `pending_confirmation` is active, affirmative responses ("yes", "haan", "sure", "proceed", "do it") execute the action; negative responses ("no", "cancel", "abort") immediately cancel and clear the state.

2. Filesystem Sandboxing (`desktop.safety.DesktopSafetyPolicy`)

- **Approved Roots Only:** Restricts write and delete operations strictly to user-owned spaces:
 - `~/Desktop`
 - `~/Documents`
 - `~/Downloads`
 - `~/Projects`
 - `~/Desktop/NOVA_WORKSPACE`
- **System Protection:** Absolute denial for system directories (`/System` , `/usr` , `/Library` , `/bin` , `/sbin` , `/etc`).
- **No Hard Overwrites:** File creation automatically generates versioned names (e.g. `file_1.txt`) unless overwrite is explicitly requested.

3. Browser Action Safety (`browser.safety.BrowserSafetyPolicy`)

- **Schema Filtering:** Blocks execution of dangerous protocol handlers (`javascript:` , `data:` , `file:`).

- **Sensitive Operations:** Flags high-risk actions (e-commerce purchase confirmations, password submissions) as requiring user interaction.

4. Immediate Cancellation

- All active execution loops (Task Agent, Computer Agent, Browser Manager) respect cancellation flags triggered by saying "*stop*", "*cancel*", or pressing Cancel in the UI.

15. 14_NOVA_PERMISSIONS.md

Source file: 14_NOVA_PERMISSIONS.md

14 — macOS Permissions & System Privileges

Overview

NOVA utilizes standard macOS system capabilities. Operating these capabilities requires specific user grants in **System Settings** $\rightarrow$ **Privacy & Security**.

Required macOS Permissions

Permission	Subsystem / Feature	Required For	Verification Mechanism
Accessibility (<code>AXUIElement</code>)	<code>core.eyes</code> , <code>core.computer_agent</code>	Inspecting window hierarchies, finding UI buttons, and injecting Quartz clicks	<code>ApplicationServices.AXIsProcessT</code>
Screen Recording (<code>CGWindowListCreateImage</code>)	<code>core.eyes</code> , <code>core.screenshot</code> , <code>core.screen_recording</code>	In-memory frame capture for OCR and visual analysis	Checked at capture time (returns empty wallpaper if denied)
Microphone	<code>voice.microphone</code> , <code>voice.manager</code>	PyAudio 16kHz audio stream capture	Checked during audio stream initialization
Automation / AppleEvents	<code>browser.engine</code> , <code>desktop.apps</code> , <code>mac_control</code>	AppleScript control of Chrome, Safari, Finder, and System Events	<code>osascript -e 'tell application "Events"...'</code>
Camera	<code>desktop.camera</code>	Photo Booth launch and	OS permission prompt on first launch

Permission	Subsystem / Feature	Required For	Verification Mechanism
		snapshot photography	
Files & Folders	desktop.files , memory.memory_manager	Reading/ writing in Desktop, Documents, and Downloads	Standard POSIX filesystem permission c

Permissions Diagnostic

The Permissions screen in the Electron UI (`ui/desktop/src/screens/PermissionsScreen.tsx`) and the CLI Doctor (`ui/doctor.py`) provide live status indicators and step-by-step setup guides for any missing macOS permissions.

16. 15_NOVA_EVENTBUS.md

Source file: 15_NOVA_EVENTBUS.md

15 — NOVA EventBus & Internal Event Stream

Overview

NOVA uses a thread-safe publish/subscribe event bus (`core.event_bus.EventBus`) as its core internal messaging backbone.



Event Catalog (`NovaEvent`)

```
class NovaEvent(str, Enum):
    APPLICATION_STARTED = "application_started"
    APPLICATION_SHUTDOWN = "application_shutdown"
    WAKE_WORD_DETECTED = "wake_word_detected"
    LISTENING_STARTED = "listening_started"
    LISTENING_PARTIAL = "listening_partial"
    LISTENING_FINISHED = "listening_finished"
    THINKING_STARTED = "thinking_started"
    RESPONSE_GENERATED = "response_generated"
    SPEAKING_STARTED = "speaking_started"
    SPEAKING_FINISHED = "speaking_finished"
    SPEECH_INTERRUPTED = "speech_interrupted"
    STATE_CHANGED = "state_changed"
    HEALTH_UPDATE = "health_update"
    COMMAND_EXECUTED = "command_executed"
    BROWSER_ACTION_STARTED = "browser_action_started"
    BROWSER_ACTION_COMPLETED = "browser_action_completed"
    BROWSER_ACTION_FAILED = "browser_action_failed"
    BROWSER_ACTION_CANCELLED = "browser_action_cancelled"
    BROWSER_TASK_STARTED = "browser_task_started"
    BROWSER_TASK_STEP_COMPLETED = "browser_task_step_completed"
    BROWSER_TASK_COMPLETED = "browser_task_completed"
    BROWSER_TASK_CANCELLED = "browser_task_cancelled"
    BROWSER_PAGE_EXTRACTED = "browser_page_extracted"
    DESKTOP_ACTION_STARTED = "desktop_action_started"
    DESKTOP_ACTION_COMPLETED = "desktop_action_completed"
    DESKTOP_ACTION_FAILED = "desktop_action_failed"
    DESKTOP_ACTION_CANCELLED = "desktop_action_cancelled"
```

```
CAMERA_CAPTURE_STARTED = "camera_capture_started"
CAMERA_CAPTURE_COMPLETED = "camera_capture_completed"
CAMERA_CAPTURE_FAILED = "camera_capture_failed"
```



EventBridge & UI Synchronization

- **Publishing:** Any subsystem publishes events via `event_bus.publish(NovaEvent.SOME_EVENT, key=val)`.
- **UI Forwarding:** `ui.backend.event_bridge.EventBridge` subscribes to these events and translates them into typed `UIEvent` objects broadcast over WebSockets to the React frontend.

16 — NOVA Frontend, Electron, & Backend Architecture

Overview

NOVA features a modern dual-UI layer: 1. **Interactive Desktop App:** Built with Electron, React 18, TypeScript, Tailwind CSS, Lucide icons, and Vite. 2. **Terminal Operations Dashboard:** Built with Python `rich`, providing an interactive activity feed, latency tracking, and command console.



Desktop Application Architecture (`ui/desktop/`)

- **Main Process (`electron/main.cjs`):** Manages native frameless window, titlebar, transparency, and IPC.
 - **Frontend Core (`src/App.tsx`):** Maintains WebSocket connection to `ws://127.0.0.1:8765`, streaming real-time avatar states and activity blocks.
 - **Interactive Avatar (`src/components/Avatar/NovaAvatar.tsx`):** Dynamic reactive orb displaying animated visual states:
 - `IDLE` : Subtle breathing glow.
 - `LISTENING` : Active audio frequency pulse.
 - `THINKING` : Rapid rotating particle halo.
 - `PLANNING` : Multi-node geometric matrix.
 - `EXECUTING` : Concentric progress wave.
 - `SPEAKING` : Expressive voice amplitude wave.
 - `SUCCESS` : Radiant emerald flash.
 - `ERROR` : Crimson warning pulse.
 - **Screens:**
 - `HomeScreen` : Quick actions, system status, active avatar.
 - `ChatScreen` : Full conversational history with dual speech/display rendering.
 - `TasksScreen` : Autonomous multi-step task tracker with step progress bars and replanning logs.
 - `VoiceScreen` : Voice V2 audio diagnostics, mic levels, and transcript logs.
 - `ActivityScreen` : Real-time audit log of all system actions, resolutions, and outcomes.
 - `SystemScreen` : Hardware metrics, AI provider status, and latency charts.
 - `PermissionsScreen` : macOS permission diagnostic checklist.
 - `SettingsScreen` : Theme, voice selection, and provider preferences.
-

UI Backend Gateway (`ui/backend/`)

- `server.py` : Asyncio HTTP REST and WebSocket server.
- `command_gateway.py` : Funnels UI text/voice commands into `NovaApplication._turn_queue`.
- `event_bridge.py` : Translates `NovaEvent` from `EventBus` into WebSocket broadcasts.

18. 17_NOVA_APPLICATIONS.md

Source file: 17_NOVA_APPLICATIONS.md

17 — macOS Application Integrations

Overview

NOVA interfaces with macOS desktop applications using native process management, AppleScript, Accessibility APIs, and CLI utilities.



Controlled Applications

Application	Launch / Open	Close / Quit	State Inspection	Interaction Mechanism
Google Chrome	<code>open -a "Google Chrome"</code>	<code>osascript quit / pkill</code>	Active tab URL, title, tab count	AppleScript + Quartz
Safari	<code>open -a Safari</code>	<code>osascript quit / pkill</code>	Active tab URL, title	AppleScript + Quartz
VS Code	<code>code <path> /</code> <code>open -a "Visual Studio Code"</code>	<code>pkill "Visual Studio Code"</code>	Active window title	CLI + AXUIElement
Finder	<code>open <path></code>	—	Selected files/folders, open path	AppleScript Finder Suite
Terminal / iTerm	<code>open -a Terminal</code>	<code>osascript quit</code>	Front window title	AppleScript + CLI
TextEdit	<code>open -a TextEdit <file></code>	<code>osascript quit</code>	Active document	AppleScript + FS
Photo Booth / Camera	<code>open -a "Photo Booth"</code>	<code>pkill "Photo Booth"</code>	Window active state	<code>CameraManager</code> / <code>AVFoundation</code>
Spotify	<code>open -a Spotify</code>	<code>osascript quit</code>	Active track & artist	AppleScript Spotify Suite
System Settings	<code>open -a "System Settings"</code>	<code>osascript quit</code>	Front window title	AppleScript

19. 18_NOVA_FILESYSTEM.md

Source file: 18_NOVA_FILESYSTEM.md

18 — NOVA Filesystem Operations & Document Architecture

Overview

Filesystem operations in NOVA are governed by `desktop.files.FileSystemManager` and `desktop.safety.DesktopSafetyPolicy`, ensuring robust file discovery, overwrite prevention, and safe trash handling.

Core Filesystem Operations

1. Multi-Root Search & Resolution

`FileSystemManager.find_files_by_name()` and `find_folders_by_name()` dynamically resolve queries across approved safe directories: - `~/Desktop` - `~/Documents` - `~/Downloads` - `~/Projects` - `~/Desktop/NOVA_WORKSPACE`

2. File & Directory Creation

- **Directory Creation:** `FileSystemManager.create_folder(name, parent, on_desktop)`.
- **File Creation:** `FileSystemManager.create_files(file_names, parent, content_map)`.
- **Safe Overwrite Protection:** Checks for existing paths and appends incremental suffixes (`name_1.ext`) unless explicit replacement is confirmed.

3. Safe Deletion & Trash Integration

- **Trash Integration:** `FileSystemManager.safe_move_to_trash(target_path)` uses macOS AppleScript or `send2trash` to move files to the system Trash bin rather than permanently unlinking (`os.remove`), preserving user recoverability.

4. AI Document Generation (`DocumentEditor`)

- `write_and_open_document(topic, destination, filename)`: Generates comprehensive, high-quality notes or essays via AI and automatically saves and launches them in TextEdit or user text editor.

20. 19_NOVA_DEPENDENCIES.md

Source file: 19_NOVA_DEPENDENCIES.md

19 — NOVA Dependencies & Environment Manifest

Overview

NOVA uses Python 3.12+ for backend execution and Node.js 18+ for desktop UI development.



Python Dependencies (`pyproject.toml` / `requirements.txt`)

Package	Version Constraint	Purpose
pydantic	<code>>=2.9.0</code>	Data modeling & runtime validation
pydantic-settings	<code>>=2.7.1</code>	Central environment configuration management
google-genai	<code>>=1.2.0</code>	Official Google Gemini SDK
groq	<code>>=0.11.0</code>	Groq AI provider SDK
openai	<code>>=1.50.0</code>	OpenRouter / OpenAI SDK client
cerebras-cloud-sdk	<code>>=1.0.0</code>	Cerebras Cloud AI provider SDK
faster-whisper	<code>>=1.0.0</code>	Localized CTranslate2 speech recognition
torch	<code>>=2.0.0</code>	Deep learning backend for VAD / Whisper
numpy	<code>>=1.24.0, <3.0.0</code>	Audio buffer and coordinate math
PyAudio	<code>>=0.2.14</code>	PortAudio low-level microphone audio streams
edge-tts	<code>>=7.0.0</code>	Microsoft Edge TTS voice synthesis
pygame	<code>>=2.6.0</code>	Audio mixer playback
pyobjc	<code>>=10.0</code>	macOS Quartz, Cocoa, CoreGraphics, Vision bindings
psutil	<code>>=5.9.0</code>	Process and memory telemetry
rich	<code>>=13.0.0</code>	Terminal Operations Dashboard & Doctor formatting
pytest	<code>>=8.0.0</code>	Automated testing suite
ruff	<code>>=0.9.0</code>	Linting and code formatting

Node.js Dependencies (`ui/desktop/package.json`)

- `electron` : `^43.4.1` — Desktop window framework
 - `react` / `react-dom` : `^18.3.1` — Component rendering
 - `vite` : `^5.4.2` — Build tool & dev server
 - `tailwindcss` : `^4.3.3` — Utility-first styling
 - `lucide-react` : `^1.16.0` — UI icons
 - `typescript` : `^5.5.3` — Static type checking
-

System Binaries (macOS)

- `ffmpeg` : Required for audio format decoding during ASR.
- `osascript` : Built-in AppleScript automation runner.
- `screencapture` : Built-in macOS screenshot and recording utility.
- `blueutil` : Optional Homebrew utility for toggling Bluetooth power state.

21. 20_NOVA_TESTING.md

Source file: 20_NOVA_TESTING.md

20 — NOVA Automated Testing Suite & Test Verification

Overview

NOVA contains an automated test suite of **881 tests** covering intent classification, provider routing, memory persistence, voice quality gating, browser automation, desktop execution, computer interaction, and task replanning.

Test Suite Execution

```
# Run full automated test suite with PYTHONPATH set to project root
PYTHONPATH=. .venv/bin/pytest --strict-markers -ra
```

Result: 881 passed, 3 warnings in 77s (100% Pass Rate).

Test Categories & Modules (tests/)

Test Module	Coverage Domain	Primary Verification
test_natural_intent_architecture.py	Intent parsing & matching	Tests Regex + semantic parsing for 35+ intents
test_task_agent_matrix.py	Multi-step task planning	Tests step generation, dependencies, execution
test_task_replanning.py	Dynamic error recovery	Tests failure interception and alternative plan injection
test_v3_5_capabilities.py	Capability registry	Tests all 21 default capability handlers and verifiers
test_browser_actions_v2.py	Browser skills & auto-scroll	Tests YouTube, Google, Amazon, and Shorts scroller
test_computer_agent_matrix.py	Visual UI interaction	Tests <code>click_element</code> , <code>type_text</code> , <code>close_popup</code>
test_environment_context.py	Environment observer	Tests front app detection and deictic pronoun resolution

Test Module	Coverage Domain	Primary Verification
<code>test_voice_quality_gate.py</code>	Voice V2 quality & ASR	Tests hallucination rejection and confidence gates
<code>test_audio_events_and_multilingual_asr.py</code>	Acoustic non-speech detection	Tests cough, sneeze, laugh, sigh detection
<code>test_backend_ui_integration.py</code>	UI server & gateway	Tests WebSocket handshake and REST endpoints
<code>test_smart_files_step4.py</code>	Filesystem safety	Tests multi-root search, folder creation, trash

22. 21_NOVA_STARTUP.md

Source file: 21_NOVA_STARTUP.md

21 — NOVA Startup, Bootstrap, & Lifecycle Management

Overview

NOVA initializes subsystems via parallel threading during bootstrap to minimize startup latency, registers teardown handlers with `ServiceLifecycle`, and ensures graceful state persistence on exit.



Startup Sequence (`main.py`)

1. Bootstrap & Logging:
 - ``setup_logging()`` configures structured logger.
 - ``lifecycle.start()`` initializes `ServiceLifecycle` registry.
 - ``EventBus`` singleton instantiated and registered.
2. Parallel Subsystem Initializations (Worker Threads):
 - └─ Thread `'nova-init-memory'`: `MemoryManager`, `VectorStore` index population.
 - └─ Thread `'nova-init-providers'`: `ProviderManager`, AI API clients.
 - └─ Thread `'nova-init-personality'`: `SystemPromptManager`, `EmotionEngine`.
 - └─ Thread `'nova-init-mac_control'`: `MacControlManager`.
 - └─ Thread `'nova-init-desktop'`: `DesktopActionManager`, `CameraManager`.
 - └─ Thread `'nova-init-browser'`: `BrowserManager`, `MacOSNativeBrowserEngine`.
 - └─ Thread `'nova-init-computer_agent'`: `ComputerAgent`, `NovaEyesManager` perception loop.
3. Voice V2 Initialization:
 - Initializes Microphone, VAD, Faster-Whisper, and Speaker.
 - Determines ``voice_available`` boolean.
4. Signal Handlers:
 - Registers `SIGINT` and `SIGTERM` handlers to trigger graceful shutdown.
5. Terminal Operations Dashboard:
 - Renders Rich terminal overview (``TerminalDashboard.render()``).
6. Context-Aware Startup Greeting:
 - Queries last active project from ``MemoryManager``.
 - Generates and delivers natural startup greeting via active AI provider.

- Publishes ``NovaEvent.APPLICATION_STARTED``.

7. Engage Background Workers:

- Starts typed console thread (``_console_input_loop``).
- Starts Voice V2 continuous listening loop.
- Enters main ``_run_event_loop()`` draining ``_turn_queue``.

Graceful Shutdown Sequence

1. Capture Session Breadcrumb:

- Writes ``last_session_summary`` (project, last topic, session duration) to Memory.

2. Stop Audio & Background Services:

- Stops VoiceListener and terminates PyAudio streams.
- Stops NovaEyes perception loop and video recorders.

3. Close External Connections:

- Shuts down AI provider client sessions and WebSocket streams.

4. Lifecycle Teardown:

- Publishes ``NovaEvent.APPLICATION_SHUTDOWN``.
- Invokes all registered teardown callbacks via ``lifecycle.shutdown()``.

22 — NOVA Security Architecture & Threat Model

Overview

NOVA adheres to a strict local-first security architecture designed to prevent secret leaks, arbitrary command execution, unauthorized filesystem access, and rogue automation.



Security Tenets

1. Secret Isolation & Zero-Leak Logging

- **Configuration Encapsulation:** All API keys are loaded into Pydantic `SecretStr` containers.
- **Log Masking:** `log_environment_debug()` and `DashboardStatsManager` actively filter and redact key, token, auth, and password patterns before printing or storing log strings.
- **Repository Hygiene:** `.env` and `credentials.json` are excluded in `.gitignore`.

2. Guarded Execution & No Blind Shell Access

- **No Arbitrary Shell Invocation:** NOVA does not expose open `eval()` or unconstrained bash execution endpoints. Actions are strictly routed through validated capability handlers (`CapabilityRegistry`).
- **Confirmation for Irreversible Actions:** Deletion of files/directories and system power off/shutdown require explicit two-turn user confirmation.

3. Localhost IPC Binding

- **Loopback Enforcement:** The UI HTTP REST and WebSocket server binds strictly to `127.0.0.1:8765`, preventing inbound remote network access from the local area network.
- **CORS Restricted:** REST endpoints enforce strict origin headers.

24. 23_NOVA_DATA_FLOW.md

Source file: 23_NOVA_DATA_FLOW.md

23 — System-Wide Data Flow & Interaction Lifecycles

Overview

This document traces complete, concrete end-to-end data flows across every major modality in NOVA.



1. Voice Interaction Turn Flow

```
[User Utterance (Speech)]
  |
  ▼
`MicrophoneManager` (16kHz Audio Stream via PyAudio)
  |
  ▼
`VoiceActivityDetector` (Silero VAD / Energy Segmentation)
  |
  ▼
`AudioEventDetector` (Detects acoustic events like coughs/laughs, strips markers)
  |
  ▼
`TranscriptionManager` (Faster-Whisper localized ASR -> Transcript & Candidates)
  |
  ▼
`TranscriptQualityGate` (Discards noise/hallucinations, passes clean text)
  |
  ▼
`NovaApplication._turn_queue` (Pushes `TurnRequest(source='voice', ...)` )
  |
  ▼
`NovaApplication._process_turn` (Pulls request)
  |
  ▼
`NaturalLanguageIntentEngine` (Classifies CanonicalIntent & extracts parameters)
  |
  ▼
[Capability Execution or AI Provider Generation]
```

```

|
▼
`ResponseOrchestrator` (Formats dual output: clean spoken audio vs rich display)
  |→ `SpeakerManager.speak()` -> Audio Output (Edge-TTS / SoundDevice)
  |→ `DashboardStatsManager.record_nova()` -> Terminal & UI WebSocket

```

2. Visual Goal Execution Flow (NOVA Eyes + Computer Agent)

```

[Visual Goal: "Click the Submit button"]
|
▼
`ComputerAgent.execute_visual_goal(goal_text)`
|
▼
`NovaEyesManager.observe_now()`
  |— Quartz Capture: `CGWindowListCreateImage` -> In-Memory `MemoryFrame`
  |— Accessibility: `AccessibilityInspector` -> `AXUIElement` hierarchy
  |— Vision OCR: Apple Silicon `VNRecognizeTextRequest` -> Text Bounding Boxes
  |
  ▼
  `EyesTargetResolver` (Maps target label to Retina (x, y) coordinates)
  |
  ▼
  `EyesInteractionManager` (Injects hardware-level Quartz `CGEvent` mouse click)
  |
  ▼
  `EyesStateVerifier` (Re-observes screen state to confirm visual outcome)
  |
  ▼
  `VisualResult(success=True, verified=True)` -> Spoken response & activity feed

```

3. Autonomous Multi-Step Task Flow

```

[Complex Goal: "Create my_app project folder and open Google Chrome"]
|
▼
`TaskPlanner.plan_goal()` -> LLM Goal Decomposition
  |— Step 1: `fs.create_folder(name="my_app")`
  |— Step 2: `app.launch(app_name="Google Chrome")`
  |

```

▼
`TaskExecutor.execute\_plan()`
└─ Step 1 Dispatch -> `CapabilityRegistry.get('fs.create\_folder').handler()`
| └─ Folder created at `~/Desktop/my\_app`
| └─ Verifier confirms directory exists -> Step 1 `SUCCESS`
└─ Step 2 Dispatch -> `CapabilityRegistry.get('app.launch').handler()`
| └─ Chrome process launched
| └─ Process check confirms running -> Step 2 `SUCCESS`
└─ Final `TaskResult(status=SUCCESS, steps\_completed=2)` -> Delivery to user

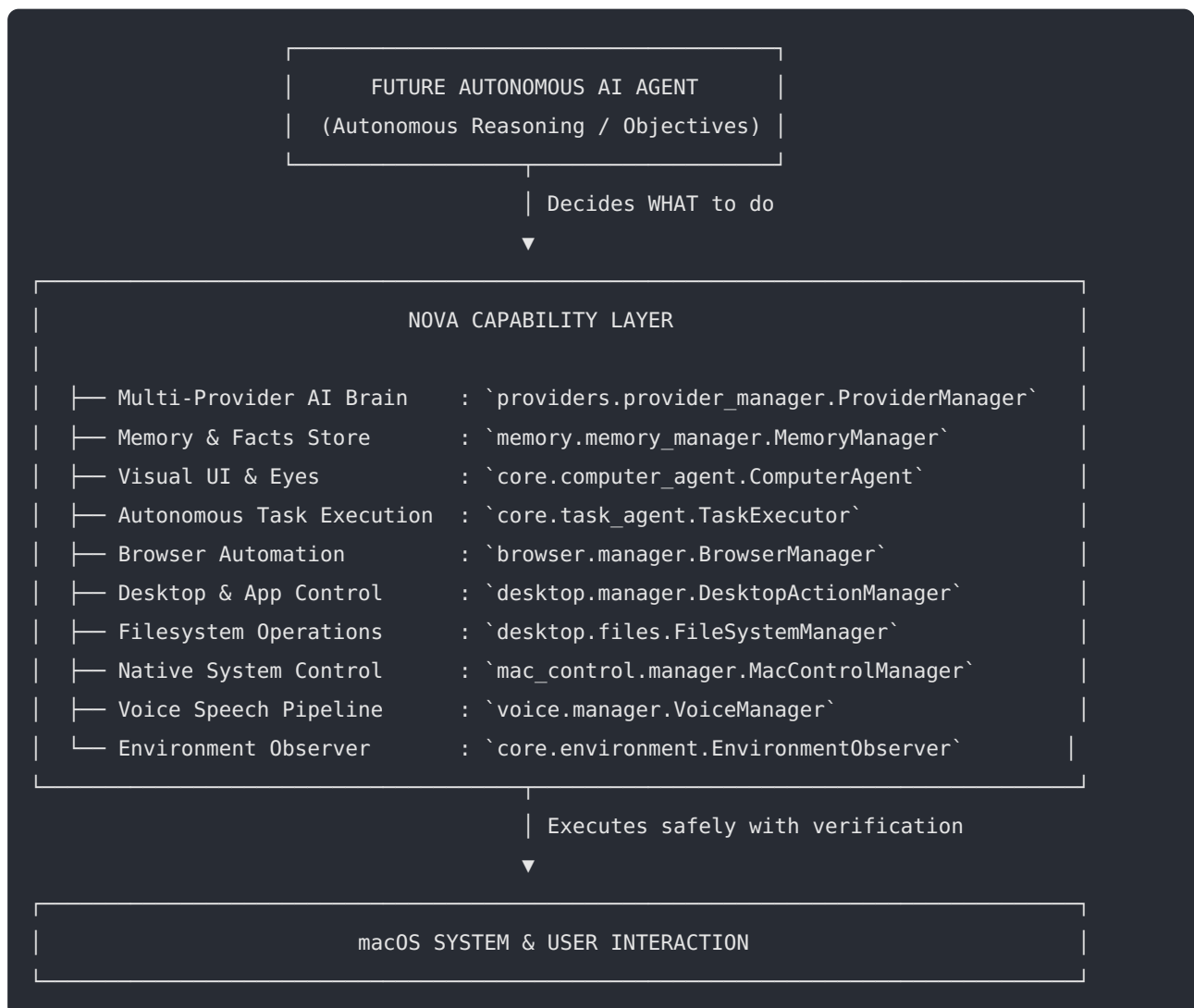
25. 24_NOVA_REUSE_GUIDE.md

Source file: 24_NOVA_REUSE_GUIDE.md

24 — Future Autonomous Agent Reuse Guide

Architecture Blueprint for Future Autonomous Agents

When building a future autonomous AI agent, student companion, or specialized reasoning brain on top of NOVA, the future agent should act as a high-level **Orchestration Brain (WHAT to do)** while delegating all perception and action to NOVA's existing **Capability Layer (HOW to execute safely)**.



Systems the Future Agent Must NOT Duplicate

1. **Do NOT write new LLM API clients:** Route queries through `ProviderManager` to retain multi-provider failover and rate limit handling.
2. **Do NOT write new browser drivers:** Invoke `BrowserManager` or register browser task steps.
3. **Do NOT build a separate mouse/keyboard controller:** Use `ComputerAgent` and `NovaEyesManager`.
4. **Do NOT implement a separate audio capture loop:** Use `VoiceManager` and `EventBus`.
5. **Do NOT bypass safety policies:** Always respect `DesktopSafetyPolicy` and `BrowserSafetyPolicy`.

26. 25_NOVA_LIMITATIONS.md

Source file: 25_NOVA_LIMITATIONS.md

25 — NOVA Known Limitations & System Constraints

Overview

This document records verified hardware, OS, and software limitations present in the current NOVA codebase.

Known Limitations & Operational Constraints

1. **Bluetooth Power Toggling:** - Querying Bluetooth status is natively supported via system commands. - Programmatically toggling Bluetooth power (on / off) on macOS requires the optional CLI utility `blueutil` (`brew install blueutil`). If absent, a warning is returned.
2. **AppleScript Browser Automation Headless Mode:** - `MacOSNativeBrowserEngine` controls real user browser windows in Chrome and Safari. It is intentionally not a headless Chromium browser; the user will see tab operations on screen.
3. **Screen OCR Language Support:** - Apple Vision framework OCR (`VNRecognizeTextRequest`) is optimized for Latin, Cyrillic, and select Asian character sets. Complex script parsing may require higher resolution or visual framing.
4. **Multi-Monitor Coordinate Offsets:** - Quartz mouse coordinates assume primary display origin (0, 0). Secondary external monitors with virtual display offsets require active window bounds mapping.
5. **Local ASR Model Execution:** - Faster-Whisper defaults to `small.en` or `base` for low-latency CPU/Metal execution. Larger models (`large-v3`) require additional RAM and GPU compute.

27. 26_NOVA_EXTENSION_POINTS.md

Source file: 26_NOVA_EXTENSION_POINTS.md

26 — NOVA Extension Points & Developer Guide

Overview

NOVA is designed to be easily extensible without modifying core orchestration loops or breaking existing test suites.

Preferred Extension Points

1. Registering New System Capabilities

New tools should be registered directly in `CapabilityRegistry` (`core.task_agent.registry`):

```
from core.task_agent.models import RiskLevel, TaskContext
from core.task_agent.registry import CapabilityDefinition, capability_registry

def _my_custom_handler(params: dict[str, Any], ctx: TaskContext) -> dict[str, Any]:
    # Implementation logic
    return {"success": True, "result": "done"}

def _my_custom_verifier(params: dict[str, Any], result: dict[str, Any], ctx: TaskContext)
-> bool:
    # Postcondition verification check
    return result.get("success", False)

capability_registry.register(
    CapabilityDefinition(
        name="custom.my_tool",
        subsystem="custom_tools",
        description="Description of what this tool accomplishes.",
        risk_level=RiskLevel.LOW,
        handler=_my_custom_handler,
        verifier=_my_custom_verifier,
        requires_confirmation=False,
    )
)
```

2. Adding New AI Providers

Subclass `providers.base_provider.BaseProvider`, implement `_generate_response_impl()`, `_stream_response_impl()`, and `health_check()`, then register with `ProviderManager`.

3. Adding New Web Site Skills

Add a new site skill class subclassing `browser.sites.base.BaseSiteSkill` inside `browser/sites/` and register it in `browser.router.BrowserActionRouter`.

4. Custom System Prompt Profiles

Register new prompt profiles with `SystemPromptManager` (`personality/system_prompt.py`) with specialized section templates and focus instructions.

28. NOVA_API_CONTRACTS.md

Source file: NOVA_API_CONTRACTS.md

NOVA Core API Contracts & Interfaces

This document specifies the public API contracts of NOVA's core modules for future autonomous agent integration.

1. Multi-Provider AI Brain (`providers.provider_manager`)

`ProviderManager`

Central gateway managing provider lifecycle, health, failover, and request dispatch.

```
class ProviderManager:
    def initialize(self) -> None:
        """Constructs and initializes default providers (Gemini, Groq, OpenRouter,
        Cerebras)."""

    def generate_response(
        self,
        prompt: str,
        system_prompt: str | None = None,
        task_type: TaskType = TaskType.GENERAL,
        mode: Literal["auto", "manual"] = "auto",
        provider_name: str | None = None,
        max_tokens: int | None = None,
    ) -> str:
        """
        Generate a non-streaming text response.

        Args:
            prompt: User text input.
            system_prompt: Optional instructions shaping the model's behavior.
            task_type: TaskType (CODING, REASONING, FAST, CHEAP, GENERAL).
            mode: "auto" selects optimal provider; "manual" enforces provider_name.
            provider_name: Required if mode="manual".
            max_tokens: Optional token generation budget.

        Returns:
            Clean text string generated by the selected model.
```

```

    Raises:
        AllProvidersFailedError: If every eligible provider fails or is unreachable.
    """

    def stream_response(
        self,
        prompt: str,
        system_prompt: str | None = None,
        task_type: TaskType = TaskType.GENERAL,
        mode: Literal["auto", "manual"] = "auto",
        provider_name: str | None = None,
    ) -> Iterator[str]:
        """Yields text chunks as they arrive from the active provider."""

    def shutdown(self) -> None:
        """Gracefully terminates underlying client sessions."""

```

2. Memory System (`memory.memory_manager`)

MemoryManager

Atomic JSON-backed persistent memory store.

```

class MemoryManager:
    def add_memory(
        self,
        category: MemoryCategory,
        key: str,
        value: Any,
        importance: int = 3,
        tags: Sequence[str] = (),
    ) -> MemoryEntry:
        """
        Persists a new memory entry atomically.

        Args:
            category: MemoryCategory enum (PROFILE, PREFERENCES, PROJECTS, CODING, etc.)
            key: Semantic descriptor label.
            value: JSON-serializable Python object or string.
            importance: Integer scale from 1 (lowest) to 5 (highest).
            tags: Sequence of classification strings.

        Returns:
            Immutable MemoryEntry instance with generated ID and timestamps.

```

```

"""

def search_memory(
    self,
    query: str,
    category: MemoryCategory | None = None,
    tags: Sequence[str] | None = None,
    min_importance: int = 1,
    exact_key: bool = False,
) -> list[MemoryEntry]:
    """Searches existing memories matching text, category, or tags."""

def delete_memory(self, memory_id: str) -> bool:
    """Deletes a memory entry by ID. Returns True if deleted."""

def shutdown(self) -> None:
    """Flushes memory store to disk and shuts down."""

```

3. Autonomous Task Agent (`core.task_agent`)

TaskPlanner

Decomposes high-level natural language goals into structured execution plans.

```

class TaskPlanner:
    def plan_goal(self, goal_text: str, context: TaskContext | None = None) -> TaskPlan:
        """
        Decomposes a goal into an ordered sequence of TaskSteps.

        Args:
            goal_text: High-level user goal (e.g. "Create a project folder and search for
            tutorials")
            context: Optional active TaskContext.

        Returns:
            TaskPlan containing goal description, estimated risk, and TaskStep list.
        """

```

TaskExecutor

Executes multi-step task plans with step-level verification and recovery.

```

class TaskExecutor:
    def execute_plan(self, plan: TaskPlan, context: TaskContext | None = None) -> TaskResu

```

```

lt:
    """
    Executes a TaskPlan step-by-step.

    Returns:
        TaskResult with overall status (SUCCESS, FAILED, CANCELLED), execution time,
        and message.
    """

    def cancel_task(self) -> None:
        """Sets internal cancellation flag, terminating active plan execution."""

    def reset_cancellation(self) -> None:
        """Resets cancellation event before starting a new task."""

```

CapabilityRegistry

Registers verified executable tools.

```

class CapabilityRegistry:
    def register(self, cap: CapabilityDefinition) -> None:
        """Registers a CapabilityDefinition with handler, verifier, and risk rating."""

    def get(self, name: str) -> CapabilityDefinition | None:
        """Retrieves a capability by unique name (e.g. 'fs.create_folder')."""

```

4. Computer Interaction & Vision (`core.computer_agent` , `core.eyes`)

ComputerAgent

Coordinates visual goal perception and execution (`SEE -> UNDERSTAND -> PLAN -> ACT -> VERIFY`).

```

class ComputerAgent:
    def execute_visual_goal(self, goal_text: str, action_hint: str | None = None) -> VisualResult:
        """Locates on-screen UI element matching goal_text and performs action."""

    def type_into_focused_or_target(
        self,
        text: str,
        target_label: str | None = None,
        submit: bool = False,
    ) -> VisualResult:

```

```

        """Types text into the active cursor focus or targeted UI input field."""

    def close_popup(self) -> VisualResult:
        """Locates and dismisses active modal popups, cookie prompts, or alerts."""

    def scroll_until_visible(self, target_text: str, max_scrolls: int = 5) ->
VisualResult:
        """Repeatedly scrolls the active viewport until target text is visible in OCR."""

    def stop_active_task(self) -> bool:
        """Interrupts and cancels active visual action."""

```

NovaEyesManager

In-memory Quartz perception engine.

```

class NovaEyesManager:
    def observe_now(self, force_ocr: bool = False) -> ScreenState:
        """
        Captures an instant in-memory Retina frame, runs AX inspection and Vision OCR.

        Returns:
            ScreenState containing active window, bounding boxes, and visible text list.
        """

```

5. Browser Automation (`browser.manager` , `browser.engine`)

BrowserManager

Coordinates browser navigation, tab management, and web skills.

```

class BrowserManager:
    def execute_command(self, text: str) -> BrowserResult | None:
        """Parses natural language browser command and executes via native engine."""

    def execute_plan(self, plan: BrowserActionPlan) -> BrowserResult:
        """Executes a structured BrowserActionPlan with safety evaluation."""

    def stop_active_task(self) -> None:
        """Terminates active browser tasks or auto-scrolling."""

```

MacOSNativeBrowserEngine

AppleScript bridge for Google Chrome and Safari.

```

class MacOSNativeBrowserEngine:
    def open_url(self, url: str) -> tuple[bool, str]:
        """Navigates the frontmost browser tab to the designated URL."""

    def open_new_tab(self, url: str = "about:blank") -> tuple[bool, int, str, str]:
        """Opens a new tab. Returns (success, tab_index, title, url)."""

    def search_current_tab(self, query: str) -> tuple[bool, int, str, str]:
        """Performs a search directly inside the currently active tab."""

    def scroll_page(self, direction: str = "down", amount: int = 500) -> bool:
        """Smoothly scrolls the active browser window."""

```

6. Environment & Reference Resolution (`core.environment`, `core.context`)

EnvironmentObserver

Real-time observer for macOS environment state.

```

class EnvironmentObserver:
    def refresh(self, force: bool = False) -> EnvironmentContext:
        """
        Captures frontmost application, window title, active URL, and Finder selection.

        Returns:
            EnvironmentContext snapshot.
        """

```

ContextResolver

Deterministic resolver for deictic natural references.

```

class ContextResolver:
    @classmethod
    def resolve_target(
        cls,
        reference_term: str,
        context: EnvironmentContext,
        target_type: str = "any",
    ) -> dict[str, Any]:
        """
        Resolves "this", "that", "the selected file", "the current tab" to a concrete Path

```

or URL.

```
Returns:
    dict with keys: 'resolved' (bool), 'target' (Path | str | None),
'target_type' (str).
    """
```

7. Event Bus (`core.event_bus`)

`EventBus`

Thread-safe publish/subscribe bus.

```
class EventBus:
    def subscribe(self, event: NovaEvent, handler: Callable[[str, dict[str, Any]], None])
-> None:
    """Registers an event listener for a NovaEvent."""

    def publish(self, event: NovaEvent, **payload: Any) -> None:
    """Publishes an event to all registered subscribers safely."""
```

29. NOVA_CAPABILITY_REGISTRY.md

Source file: NOVA_CAPABILITY_REGISTRY.md

NOVA Capability Registry & Execution Catalog

This document catalogs all verified capabilities built into NOVA, mapping each capability from user intent down to OS-level execution and verification.

Comprehensive Capability Catalog

1. Filesystem Capabilities

Capability ID	Description	Intent	Executor	OS / Platform
<code>fs.create_folder</code>	Create a folder on Desktop or designated directory	<code>CREATE_FOLDER</code>	<code>FileSystemManager.create_folder</code>	path
<code>fs.create_file</code>	Create source code or document file	<code>CREATE_FILE</code>	<code>FileSystemManager.create_files</code>	path
<code>fs.open_folder</code>	Open directory in Finder	<code>OPEN_FOLDER</code>	<code>FileSystemManager</code>	subpath]
<code>fs.safe_move_to_trash</code>	Move file or directory to macOS Trash	<code>DELETE_ITEM</code>	<code>FileSystemManager.safe_move_to_trash</code>	osascript

2. Document & Code Capabilities

Capability ID	Description	Intent	Executor
<code>doc.write_and_open</code>	Generate AI document and open in editor	<code>GENERATE_AND_WRITE_DOCUMENT</code>	<code>DocumentEditor.write_and_open</code>
<code>code.scaffold_project</code>		<code>SCAFFOLD_PROJECT</code>	<code>CodeFileManager</code>

Capability ID	Description	Intent	Executor
	Generate code project scaffolding		

3. Application Control

Capability ID	Description	Intent	Executor	OS / API	Verification
<code>app.launch</code>	Launch any macOS application	<code>LAUNCH_APP</code>	<code>AppLauncher.launch</code>	<code>subprocess.run(["open", "-a", app])</code>	Process active check
<code>app.close</code>	Close or terminate macOS application	<code>CLOSE_APP</code>	<code>AppLauncher.close</code>	<code>osascript quit / pkill</code>	Process terminated

4. Browser Capabilities

Capability ID	Description	Intent	Executor
<code>browser.open_site</code>	Navigate to website URL	<code>OPEN_SITE</code>	<code>MacOSNativeBrowserEngine.open_url</code>
<code>browser.search_web</code>	Search Google for query	<code>SEARCH_WEB</code>	<code>GoogleSkill.search</code>
<code>browser.search_site</code>	Search inside Amazon/ GitHub/ YouTube	<code>SEARCH_SITE</code>	<code>SiteSkill.search</code>
<code>browser.open_new_tab</code>	Open a new browser tab	<code>OPEN_NEW_TAB</code>	<code>MacOSNativeBrowserEngine.open_new</code>
<code>browser.search_current_tab</code>	Search within current tab	<code>SEARCH_CURRENT_TAB</code>	<code>MacOSNativeBrowserEngine.search_cu</code>
<code>browser.scroll</code>	Scroll page up / down / top / bottom	<code>SCROLL_DOWN / SCROLL_UP</code>	<code>MacOSNativeBrowserEngine.scroll_pa</code>
<code>browser.auto_shorts</code>	Continuous YouTube Shorts scrolling	<code>START_AUTO_SHORTS</code>	<code>YouTubeShortsScroller</code>

Capability ID	Description	Intent	Executor
<code>browser.research</code>	Multi-step deep web research	<code>RESEARCH_TOPIC</code>	<code>BrowserTaskPlanner.execute_research</code>

5. Visual UI & Screen Perception (NOVA Eyes)

Capability ID	Description	Intent	Executor
<code>visual.click_element</code>	Click UI button/link by label	<code>CLICK_UI_ELEMENT</code>	<code>ComputerAgent.execute_visual_goal</code>
<code>visual.type_text</code>	Type text into input field	<code>TYPE_UI_TEXT</code>	<code>ComputerAgent.type_into_focused_or_target</code>
<code>visual.close_popup</code>	Close active modal dialog/popup	<code>CLOSE_POPUP</code>	<code>ComputerAgent.close_popup</code>
<code>visual.scroll_until</code>	Scroll until text is visible	<code>SCROLL_UNTIL_VISIBLE</code>	<code>ComputerAgent.scroll_until_visible</code>
<code>visual.observe_screen</code>	Summarize/explain screen	<code>WHAT_AM_I_LOOKING_AT</code>	<code>ComputerAgent.explain_screen_content</code>

6. System & Hardware Control

Capability ID	Description	Intent	Executor
<code>system.volume</code>	Adjust or mute system volume	<code>CONTROL_VOLUME</code>	<code>mac_control.actions.volume</code>
<code>system.brightness</code>	Adjust display brightness	<code>CONTROL_BRIGHTNESS</code>	<code>mac_control.actions.brightness</code>
<code>system.wifi</code>	Toggle or query Wi-Fi	<code>CONTROL_WIFI</code>	<code>mac_control.actions.wifi</code>
<code>system.bluetooth</code>	Query or toggle Bluetooth	<code>CONTROL_BLUETOOTH</code>	<code>mac_control.actions.bluetooth</code>

Capability ID	Description	Intent	Executor
<code>system.lock_screen</code>	Lock macOS screen	<code>LOCK_SCREEN</code>	<code>mac_control.actions.system.lock_scre</code>
<code>system.shutdown</code>	Shut down host machine	<code>SYSTEM_SHUTDOWN</code>	<code>mac_control.actions.system</code>
<code>camera.open_camera</code>	Open camera viewfinder	<code>OPEN_CAMERA</code>	<code>CameraManager.open_camera</code>
<code>camera.take_photo</code>	Take snapshot photo	<code>TAKE_PHOTO</code>	<code>CameraManager.capture_photo</code>
<code>screen.capture</code>	Take full screenshot	<code>SCREEN_CAPTURE</code>	<code>ScreenshotService.capture_full_scre</code>
<code>screen.record_start</code>	Start background screen record	<code>START_SCREEN_RECORDING</code>	<code>ScreenRecordingManager.start_recordi</code>
<code>screen.record_stop</code>	Stop background screen record	<code>STOP_SCREEN_RECORDING</code>	<code>ScreenRecordingManager.stop_recording</code>
<code>clipboard.get</code>	Read clipboard text	<code>GET_CLIPBOARD</code>	<code>mac_control.actions.clipboard.get_cli</code>
<code>clipboard.set</code>	Write clipboard text	<code>SET_CLIPBOARD</code>	<code>mac_control.actions.clipboard.set_cli</code>
<code>clipboard.clear</code>	Clear clipboard text	<code>CLEAR_CLIPBOARD</code>	<code>mac_control.actions.clipboard.clear_c</code>

30. NOVA_COMPLETE_CONTEXT.md

Source file: NOVA_COMPLETE_CONTEXT.md

NOVA Master Architecture & System Context

Target Audience: Autonomous AI Agent Builders, Core Engineers, System Architects

Status: Verified Active Codebase Reference (NOVA v3.5 / Core v0.1.0)

Test Suite Status: 881 Tests Passed (100% Green)

1. Executive Summary & Philosophy

NOVA is a modular, voice-first, multimodal personal AI operating system tailored specifically for macOS. It bridges high-level reasoning, perceptual awareness (vision, audio events, OS state), and concrete system execution (desktop actions, browser automation, file operations, hardware control) into a unified, responsive companion.

Core Architectural Pillars:

1. **Perception-Action Loop (EYES & HANDS):** Real-time awareness of frontmost applications, browser tabs, Finder selections, and screen state enables deterministic resolution of natural deictic references ("this file", "that tab", "here").
2. **Layered Intent Architecture:** Natural language inputs are classified through deterministic regex matching and semantic entity extraction before falling back to LLM reasoning, ensuring sub-10ms response times for common actions.
3. **Resilient AI Provider Failover:** ProviderManager automatically balances between Gemini, Groq, OpenRouter, and Cerebras with zero downtime if any provider experiences rate limits or network degradation.
4. **Local-First Safety & Isolation:** Destructive operations (deletion, system shutdown) require explicit user confirmation. Computer vision and screen inspection occur 100% in-memory without unneeded disk dumps or cloud video streaming.

2. Global Repository Structure

```
NOVA_SETUP/  
├─ config/  
│   └─ settings.py           # Pydantic Settings v2 central configuration schema  
├─ core/
```

```

|   └─ computer_agent.py      # OBSERVE -> UNDERSTAND -> PLAN -> ACT -> VERIFY engine
|   └─ constants.py          # System-wide static constants & timeouts
|   └─ context.py            # RecentInteractionContext & ResolvedEntity tracker
|   └─ environment.py        # EnvironmentObserver & ContextResolver (macOS
AppleScript)
|   └─ event_bus.py          # Application-wide thread-safe publish/subscribe
EventBus
|   └─ exceptions.py         # Canonical exception hierarchy (AIProviderError, etc.)
|   └─ lifecycle.py          # Global ServiceLifecycle registry for start/stop hooks
|   └─ logger.py             # Structured logging configuration
|   └─ paths.py              # Standardized directory paths (data, logs, memory,
media)
|   └─ registry.py           # Global service singleton registry
|   └─ response_cleaner.py    # LLM output sanitization (markdown/thinking stripping)
|   └─ screen_recording.py    # ScreenRecordingManager (AVFoundation / ffmpeg /
screenshot)
|   └─ screenshot.py         # ScreenshotService (Desktop/NOVA Screenshots)
|   └─ eyes/
|       └─ accessibility.py   # macOS AXUIElement accessibility tree traversal
|       └─ capture.py         # CGWindowListCreateImage screen grabber
|       └─ interaction.py     # Quartz CGEvent mouse/keyboard injector
|       └─ manager.py         # NovaEyesManager background polling & proactive checks
|       └─ planner.py         # EyesTargetResolver (spatial/label mapping)
|       └─ proactive.py       # ProactiveAssistanceEngine (error/opportunity detector)
|       └─ state.py           # ScreenState, ProductCandidate, PageCategory data
models
|   └─ verifier.py           # Pre/post-action UI delta verifier
|   └─ vision_ocr.py         # Apple VNRecognizeTextRequest / OCR bridge
|   └─ task_agent/
|       └─ executor.py        # TaskExecutor (step dispatch, recovery, replanning)
|       └─ models.py         # TaskGoal, TaskPlan, TaskStep, TaskContext data models
|       └─ planner.py        # TaskPlanner (LLM goal decomposition into atomic steps)
|       └─ registry.py       # CapabilityRegistry (catalog of verified system
capabilities)
|   └─ replanner.py          # DynamicReplanner (automatic failure recovery &
alternative plans)
|   └─ visual/
└─ browser/
    └─ auto_scroll.py        # YouTube Shorts & infinite scroll background driver
    └─ context.py            # BrowserContextManager (active tabs, page state)
    └─ engine.py             # MacOSNativeBrowserEngine (AppleScript Chrome/Safari
bridge)
    └─ extractor.py          # Webpage article & structured text extractor
    └─ manager.py            # BrowserManager orchestrator
    └─ models.py             # BrowserActionPlan, BrowserResult, ActionType
    └─ parser.py             # BrowserIntentParser (rule-based action parser)

```

```

|   ├── planner.py           # BrowserTaskPlanner (multi-step web research)
|   ├── router.py           # BrowserActionRouter (dispatches to site skills)
|   ├── safety.py           # BrowserSafetyPolicy (URL validation & confirmation
checks)
|   ├── sessions.py         # BrowserSessionManager (cookie & session preservation)
|   └── sites/              # Domain-specific web skills
|       ├── amazon.py       # Amazon search, product filtering, price checking
|       ├── github.py       # GitHub repo search & code navigation
|       ├── google.py       # Google Search query builder & result extractor
|       └── youtube.py      # YouTube search, video playback, auto-play next, Shorts
└── desktop/               # Desktop & filesystem actions
    ├── apps.py            # AppLauncher (macOS open/kill/pkill bridge)
    ├── camera.py          # CameraManager (Photo Booth, snapshot capture)
    ├── code.py            # CodeFileManager (smart code editing & project
scaffolding)
    ├── context.py         # DesktopContext (active project, folder paths)
    ├── editor.py          # DocumentEditor (AI document generation & opening)
    ├── files.py           # FileSystemManager (multi-root search, safe trash,
folder creation)
    ├── manager.py         # DesktopActionManager orchestrator
    ├── models.py          # DesktopActionPlan, DesktopActionResult
    ├── parser.py          # DesktopIntentParser
    ├── planner.py         # DesktopTaskPlanner
    └── safety.py          # DesktopSafetyPolicy (path validation & sandbox roots)
└── intent/               # Natural Language Intent & Routing Engine
    ├── disambiguator.py   # Disambiguation engine for ambiguous commands
    ├── engine.py          # NaturalLanguageIntentEngine (3-tier intent
classification)
    ├── entity_resolver.py # Entity & parameter extraction (apps, targets, queries)
    ├── fallback.py        # Conversational LLM fallback wrapper
    ├── matcher.py         # Fast regex pattern matcher for canonical intents
    ├── models.py          # CanonicalIntent enum, StructuredAction dataclass
    ├── normalizer.py      # Spoken text normalizer (ASR typo correction, numbers)
    └── router.py          # RoutingDomain mapper & structured plan converters
└── mac_control/          # Native macOS system controls
    ├── manager.py         # MacControlManager orchestrator
    ├── models.py          # MacCommand, MacResult, CommandCategory
    ├── parser.py          # MacControlParser
    ├── router.py          # MacControlRouter
    └── actions/           # Subsystem executors
        ├── bluetooth.py   # Bluetooth status & blueutil toggle bridge
        ├── brightness.py  # Display brightness query & adjustment
        ├── clipboard.py   # macOS pbcopy/pbpaste bridge
        ├── finder.py      # Finder reveal & file selection
        ├── media.py       # System media key simulation
        └── network.py     # Network interface diagnosis

```

```

|   ├── screenshot.py      # screenshot CLI executor
|   ├── system.py          # Lock screen & shutdown handlers
|   ├── volume.py          # System audio volume query, set, mute, unmute
|   └── wifi.py            # networksetup Wi-Fi power toggle & SSID query
└── memory/                # Persistent & Semantic Memory
    ├── memory_manager.py  # MemoryManager (atomic JSON storage, CRUD, categories)
    └── vector_store.py    # VectorStore (ChromaDB semantic search with mock
fallback)
└── personality/          # Character & Response Generation
    ├── emotion_engine.py  # ConversationMode detector (Coding, Study, Casual,
etc.)
    ├── response_orchestrator.py # Dual-channel response formatting (TTS speech vs Rich
display)
    └── system_prompt.py   # Modular SystemPromptManager & prompt profiles
└── providers/            # Multi-provider AI Gateway
    ├── base_provider.py   # BaseProvider abstract base class with stats/timing
    ├── cerebras.py        # CerebrasCloudProvider (qwen-3.8-27b)
    ├── gemini.py          # GeminiProvider (google-genai SDK, gemini-2.5-flash)
    ├── groq.py            # GroqProvider (groq SDK, compound-mini)
    ├── openrouter.py      # OpenRouterProvider (openai SDK, gpt-4o-mini)
    └── provider_manager.py # ProviderManager routing priorities & automatic
failover
└── ui/                   # Frontend & Visualization
    ├── doctor.py          # Rich CLI diagnostic doctor suite
    ├── health_checker.py  # DashboardStatsManager activity & latency tracker
    ├── terminal_dashboard.py # Rich terminal activity feed & telemetry UI
    ├── backend/           # Async WebSocket & HTTP REST server
    │   ├── command_gateway.py # UI -> Core turn queue command bridge
    │   ├── event_bridge.py  # EventBus -> WebSocket client broadcaster
    │   ├── launcher.py     # Background subprocess launcher for Electron/UI
    │   ├── models.py       # Pydantic models for UI events, commands, avatar state
    │   └── server.py        # Python asyncio HTTP REST & WebSocket server
(127.0.0.1:8765)
    ├── desktop/           # Electron + React 18 + Vite + Tailwind desktop app
    │   ├── electron/main.cjs # Electron main process (frameless window, IPC)
    │   └── src/            # React UI components (Avatar, Screens, TopBar, Sidebar)
└── voice/                 # Voice V2 Audio Pipeline
    ├── audio_events.py    # Non-speech acoustic detector (cough, laugh, sneeze,
sigh)
    ├── commands.py        # Local keyword recognizer & fast voice command router
    ├── diagnostics.py     # Audio hardware & PortAudio diagnostic checks
    ├── listener.py        # VoiceListener background audio recording thread
    ├── manager.py         # VoiceManager orchestrator
    ├── microphone.py      # PyAudio stream manager
    ├── preprocessing.py   # RMS normalization & silence trimming
    └── quality.py         # TranscriptQualityGate (hallucination & noise filter)

```

```

|   ├── speaker.py           # Edge-TTS / Pygame / Kokoro audio playback
|   ├── state.py             # VoiceState, TranscriptionResult, SpeechOutputResult
|   ├── transcriber.py       # Faster-Whisper localized speech-to-text
|   └── vad.py               # Silero VAD energy & voice activity detector
└── main.py
# NovaApplication entry point, turn queue loop, graceful shutdown

```

3. Subsystem Deep Dives

3.1 Orchestration Loop (`main.py`)

`NovaApplication` acts as the primary coordinator: - **Startup:** Initializes subsystems in parallel threads (`_initialize_memory` , `_initialize_providers` , `_initialize_personality` , `_initialize_mac_control` , `_initialize_desktop` , `_initialize_browser` , `_initialize_computer_agent`). - **Turn Queue (`_turn_queue`):** Consumes `TurnRequest` instances from voice or typed input. - **Turn Execution Pipeline:** 1. Environment snapshot capture via `EnvironmentObserver.refresh()` . 2. Pending confirmation resolution (e.g. confirming file deletion or shutdown). 3. Local voice command intercept (e.g. immediate local shutdown or cancellation). 4. Layered intent parsing via `NaturalLanguageIntentEngine` . 5. Action dispatch to the designated domain (`TASK_AGENT` , `BROWSER` , `DESKTOP_APP` , `FILESYSTEM` , `DOCUMENT_WRITING` , `SYSTEM` , `VISUAL`). 6. Conversational fallback via `ProviderManager.generate_response()` with personality and memory injection. 7. Response orchestration via `ResponseOrchestrator` separating clean spoken audio for TTS from rich formatted text for UI.

3.2 AI Providers (`providers/`)

- **Base Interface:** `BaseProvider` enforces `generate_response()` , `stream_response()` , `health_check()` , `initialize()` , and `shutdown()` .
- **Automatic Routing Priority:**
- **CODING:** Gemini $\rightarrow$ Groq $\rightarrow$ OpenRouter $\rightarrow$ Cerebras
- **REASONING:** Gemini $\rightarrow$ OpenRouter $\rightarrow$ Groq $\rightarrow$ Cerebras
- **FAST:** Groq $\rightarrow$ Cerebras $\rightarrow$ Gemini $\rightarrow$ OpenRouter
- **CHEAP:** Cerebras $\rightarrow$ Groq $\rightarrow$ Gemini $\rightarrow$ OpenRouter
- **GENERAL:** Gemini $\rightarrow$ Groq $\rightarrow$ OpenRouter $\rightarrow$ Cerebras
- **Failover:** If a provider raises `AIProviderError` (e.g. rate limit 429, timeout, network failure), `ProviderManager` automatically catches it, records the error, and falls back to the next provider in the priority list.

3.3 Voice V2 Subsystem (`voice/`)

- **Capture:** `MicrophoneManager` streams 16kHz mono audio via `PyAudio` .
- **VAD:** `VoiceActivityDetector` uses energy thresholding and Silero VAD parameters to segment utterances.

- **ASR:** `TranscriptionManager` runs localized `faster-whisper` (default model: `small.en / base`), generating text hypotheses, confidence scores, and alternative candidates.
- **Audio Event Detection:** `AudioEventDetector` intercepts acoustic patterns (coughs, sneezes, laughter, sighs) to trigger caring responses without conversational LLM overhead.
- **TTS:** `SpeakerManager` generates speech using Microsoft `edge-tts` (voice: `hi-IN-SwaraNeural` or customizable) or native `pyttsx3` / Kokoro fallback.

3.4 Perception & Computer Control (`core/eyes/` , `core/computer_agent.py`)

- **Observation:** `NovaEyesManager` captures Retina screen frames directly into memory using Quartz `CGWindowListCreateImage` and extracts text bounding boxes with Apple Vision OCR (`VNRecognizeTextRequest`).
- **Accessibility:** `AccessibilityInspector` traverses macOS `AXUIElement` trees to identify buttons, text inputs, windows, and modal popups.
- **ComputerAgent Loop:** SEE -> UNDERSTAND -> PLAN -> ACT -> OBSERVE AGAIN -> VERIFY .
- **Quartz Interaction:** Injects hardware-level mouse clicks, scrolling, and keyboard events using macOS `CGEventCreateMouseEvent` and `CGEventCreateKeyboardEvent` .

3.5 Autonomous Task Agent V3 (`core/task_agent/`)

- **Goal Decomposition:** `TaskPlanner` uses LLM reasoning to decompose complex goals into discrete `TaskStep` definitions with dependencies.
- **Execution:** `TaskExecutor` executes each step through `CapabilityRegistry` , verifying postconditions via registered verifiers.
- **Dynamic Replanning:** `DynamicReplanner` intercepts step failures, analyzes the error context and current screen state, and generates adaptive alternative steps (up to `max_replans=3`).

3.6 UI & Communication Bridge (`ui/`)

- **Backend Server:** `NovaUIServer` runs an `asyncio` HTTP REST and WebSocket server on `127.0.0.1:8765` .
- **Command Gateway:** `CommandGateway` funnels UI inputs directly into `NovaApplication._turn_queue` .
- **Event Bridge:** `EventBridge` subscribes to `NovaEvent` on the `EventBus` and broadcasts structured JSON payloads to all connected WebSocket clients.
- **Frontend:** React 18 + TypeScript + Vite desktop client wrapped in Electron with interactive Avatar states (`IDLE` , `LISTENING` , `THINKING` , `PLANNING` , `EXECUTING` , `SPEAKING` , `SUCCESS` , `ERROR`).

4. Invariants for Future Autonomous Development

When building future autonomous capabilities on top of NOVA: 1. **Never reinvent browser automation:** Reuse `BrowserManager` and `MacOSNativeBrowserEngine` . 2. **Never reinvent mouse/keyboard control:** Reuse `ComputerAgent` and `EyesInteractionManager` .

3. **Never bypass safety policies:** Always respect `DesktopSafetyPolicy` and `BrowserSafetyPolicy`. 4. **Never create secondary AI provider clients:** Route all model queries through `ProviderManager`. 5. **Never expose or hardcode secrets:** Adhere strictly to `config.settings.settings`.

31. NOVA_NON_BREAKING_CONTRACT.md

Source file: NOVA_NON_BREAKING_CONTRACT.md

NOVA Non-Breaking Architectural Contract

This document outlines the strict compatibility and safety invariants that all future autonomous agent development must preserve.

1. Secrets & Environment Configuration Invariants

- **Never Hardcode Secrets:** All external API keys (`GEMINI_API_KEY` , `GRQ_API_KEY` , `OPENROUTER_API_KEY` , `CEREBRAS_API_KEY` , `TAVILY_API_KEY` , `ELEVENLABS_API_KEY` , etc.) must be accessed exclusively through `config.settings.settings` .
 - **Git Protection:** `.env` and `credentials.json` must remain ignored by `.gitignore` .
 - **Masking:** Logs and UI telemetry must always mask sensitive credentials (`SecretStr.get_secret_value()` should never appear in plain log sinks).
-

2. Safety & Sandboxing Invariants

- **Destructive Operation Gates:**
 - File/folder deletion (`fs.safe_move_to_trash` / `DELETE_ITEM`) must always require explicit confirmation unless running in pre-authorized headless sandbox mode.
 - System shutdown (`system.shutdown`) must always require confirmation.
 - **Filesystem Confinement:** File writing, project scaffolding, and folder creation must remain within approved user directories (`Desktop` , `Documents` , `Downloads` , `Projects` , `DESKTOP_WORKSPACE_DIR`). Never write arbitrary files outside these paths.
 - **URL Filtering:** Malicious schemas (`javascript:` , `data:` , `file:`) must remain blocked by `BrowserSafetyPolicy` .
-

3. Concurrency & EventBus Invariants

- **Thread Safety:** `EventBus` , `MemoryManager` , `RecentInteractionContext` , and `NovaEyesManager` must remain protected by thread-safe locks (`threading.RLock` / `threading.Lock`).
- **Non-Blocking Main Loop:** Turn processing in `NovaApplication` and UI server WebSocket broadcasting must never block the main event loop or audio ingestion thread.

- **Cancellation Propagation:** Calling `cancel_task()` or `stop_active_task()` must cleanly propagate cancellation flags to active `TaskExecutor`, `ComputerAgent`, `BrowserManager`, and `DesktopActionManager` instances.
-

4. IPC & Port Invariants

- **UI Gateway Port:** Local WebSocket and REST server must default to `127.0.0.1:8765`.
 - **Payload Schema:** WebSocket event payloads must conform to `ui.backend.models.UIEvent` with `event_type`, `avatar_state`, and `data` fields to avoid breaking the React desktop interface.
-

5. Testing & Verification Invariants

- **Test Integrity:** Existing pytest test suite (881 passing tests) must remain 100% green.
- **No Direct Mock Pollution:** Tests must not modify global singletons (`registry`, `capability_registry`) permanently without restoring them in teardown fixtures.

32. NOVA_REUSE_MATRIX.md

Source file: NOVA_REUSE_MATRIX.md

NOVA Reuse Matrix

This matrix establishes the engineering policy for how future autonomous agents and modules must interact with existing NOVA subsystems.



Subsystem Reuse & Integration Table

NOVA Subsystem	Architecture Action	Preferred Entry Point / Class	Do NOT Duplicate	Rationale & Rules
AI Providers	REUSE DIRECTLY	<code>providers.provider_manager.ProviderManager</code>	❌ DO NOT CREATE new LLM clients	Centralizes to limiting, and a fallback across OpenRouter, O
Memory System	REUSE DIRECTLY	<code>memory.memory_manager.MemoryManager</code>	❌ DO NOT CREATE separate storage	Atomic JSON s category part (coding , pr conversatio quarantine re
Semantic Search	WRAP / ADAPT	<code>memory.vector_store.VectorStore</code>	❌ DO NOT CREATE vector engines	Provides Chro document ret memory fallba
Voice V2 (STT/TTS)	REUSE DIRECTLY	<code>voice.manager.VoiceManager</code>	❌ DO NOT CREATE new audio pipelines	Integrates Py VAD, Faster-W Edge-TTS into synchronized
Audio Events	REUSE DIRECTLY	<code>voice.audio_events.AudioEventDetector</code>	❌ DO NOT REBUILD	Acoustic dete sneezes, laug with empathic
Browser Engine	REUSE DIRECTLY	<code>browser.engine.MacOSNativeBrowserEngine</code>	❌ DO NOT CREATE browser drivers	Native AppleS Chrome/Safar DOM extractio
Browser Manager	REUSE DIRECTLY	<code>browser.manager.BrowserManager</code>	❌ DO NOT REBUILD browser routers	Coordinates b (YouTube, Am GitHub, gener step research
Auto-Scroll Engine	REUSE DIRECTLY	<code>browser.auto_scroll.YouTubeShortsScroller</code>	❌ DO NOT REBUILD	Non-blocking scrolling for Y and web feed

NOVA Subsystem	Architecture Action	Preferred Entry Point / Class	Do NOT Duplicate	Rationale & Rules
Computer Agent	WRAP / ADAPT	<code>core.computer_agent.ComputerAgent</code>	✗ DO NOT CREATE mouse/ keyboard code	Full SEE -> U PLAN -> ACT loop using Qu Accessibility.
NOVA Eyes (Perception)	REUSE DIRECTLY	<code>core.eyes.manager.NovaEyesManager</code>	✗ DO NOT STREAM screen to cloud	In-memory Re capture (CGWindowLi and Apple Vis
Environment Observer	REUSE DIRECTLY	<code>core.environment.EnvironmentObserver</code>	✗ DO NOT REBUILD OS state monitors	Real-time from window title, Finder selectio AppleScript.
Context Resolver	REUSE DIRECTLY	<code>core.environment.ContextResolver</code>	✗ DO NOT REBUILD pronoun logic	Deterministic "this", "that", active files, ta
Recent Interaction Context	REUSE DIRECTLY	<code>core.context.RecentInteractionContext</code>	✗ DO NOT CREATE turn history buffers	Tracks bounde interaction his targets, and s
Task Planner	EXTEND	<code>core.task_agent.planner.TaskPlanner</code>	✗ DO NOT REBUILD step models	LLM-driven ta decomposition TaskStep ch agents can re prompts.
Task Executor	REUSE DIRECTLY	<code>core.task_agent.executor.TaskExecutor</code>	✗ DO NOT REBUILD execution runners	Executes plan with timeout l cancellation o post verificati
Task Replanner	REUSE DIRECTLY	<code>core.task_agent.replanner.DynamicReplanner</code>	✗ DO NOT REBUILD recovery loops	Automatically recovery plan fails based on screen inspec
Capability Registry	EXTEND	<code>core.task_agent.registry.CapabilityRegistry</code>	✗ DO NOT HARDCODE capabilities	Central regist executable to browser.* , system.*).
Desktop Actions	REUSE DIRECTLY	<code>desktop.manager.DesktopActionManager</code>	✗ DO NOT REBUILD app launchers	Manages app (AppLaunche (CameraMana docs (Docume
Filesystem Manager	REUSE DIRECTLY	<code>desktop.files.FileSystemManager</code>	✗ DO NOT WRITE	Multi-root sea integration, o

NOVA Subsystem	Architecture Action	Preferred Entry Point / Class	Do NOT Duplicate	Rationale & Rules
			unsafe file tools	protection, and creation.
macOS Control	REUSE DIRECTLY	<code>mac_control.manager.MacControlManager</code>	✗ DO NOT REBUILD system scripts	Native controls for brightness, W clipboard, local shutdown.
EventBus	REUSE DIRECTLY	<code>core.event_bus.EventBus</code>	✗ DO NOT CREATE secondary buses	Central thread to subscribe bus voice, task, and
UI Server & Gateway	REUSE DIRECTLY	<code>ui.backend.server.NovaUIServer</code>	✗ DO NOT CREATE new web servers	Async WebSockets REST gateway on 127.0.0.1:8 and core.
Safety Policies	REUSE DIRECTLY	<code>desktop.safety.DesktopSafetyPolicy</code>	✗ NEVER BYPASS safety checks	Sandboxing, and action confirmation for dangerous UR